IS 2110: Data Structures and Algorithms II

Graph Algorithms

Lecture 05-07: Graph Representations

By:

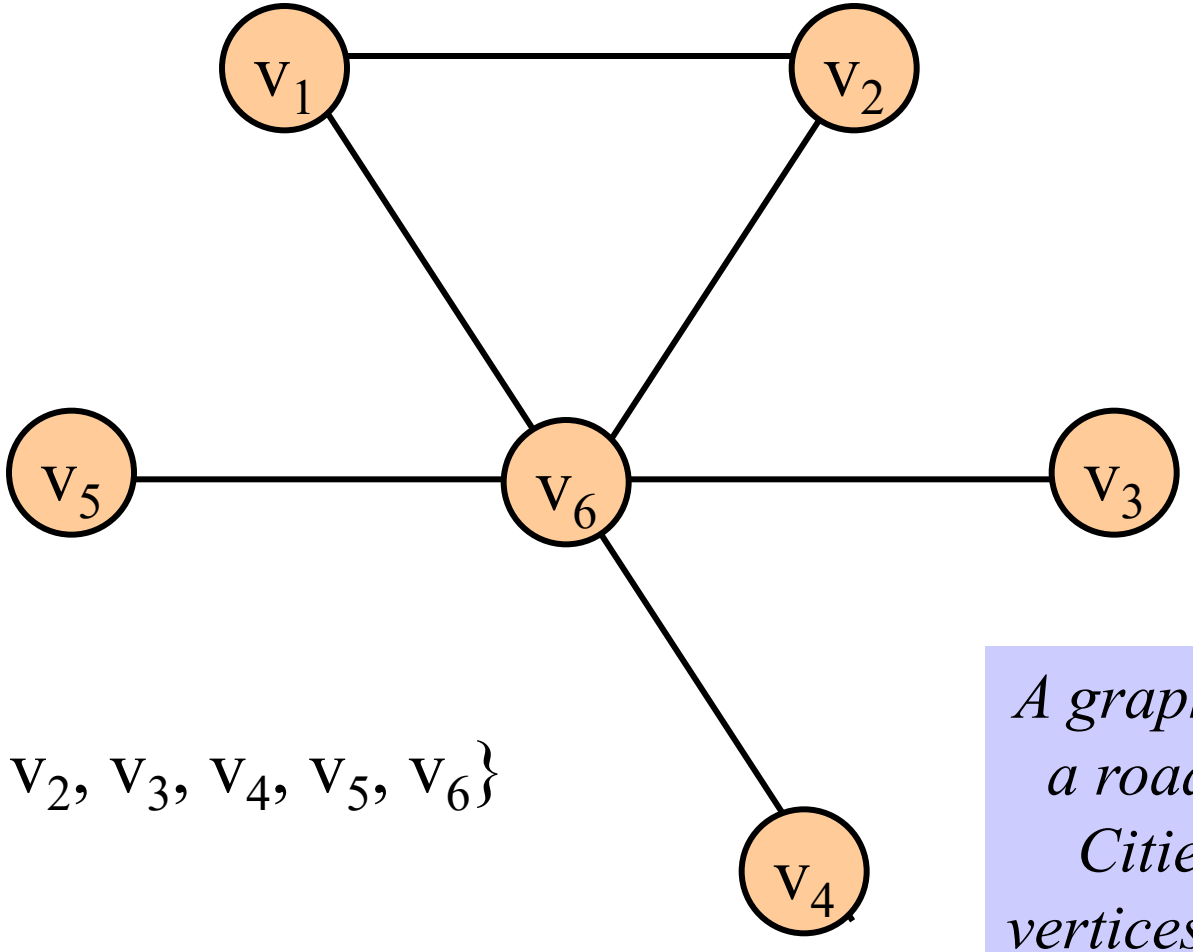
Dr. Kasun Karunanayaka

(Most of the slides are adopted from Prof. Jeevani Goonetillake's presentation)

The Graph Data Structure

- A **Graph** is set of items connected by *edges*. Each item is called a *vertex* or *node*.

- A graph **G = (V,E)** is a set of vertices (V) and a binary relation between vertices (set of edges - E).

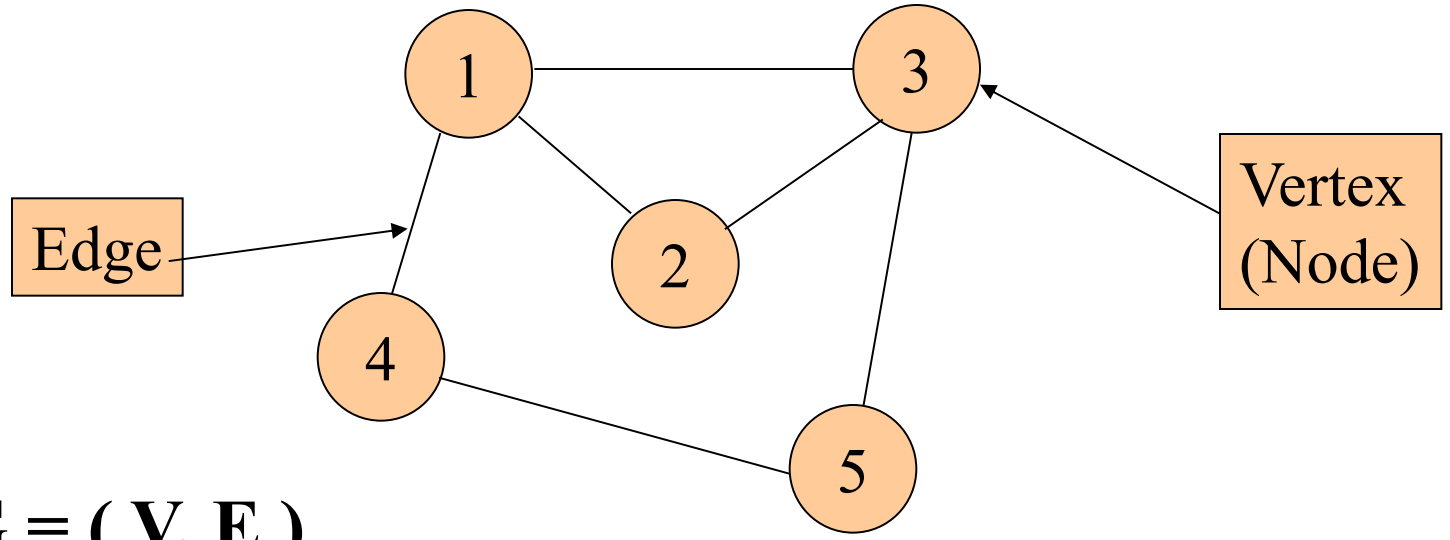


$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$

$E = \{(v_1, v_2), (v_1, v_6), (v_2, v_6), (v_3, v_6), (v_4, v_6), (v_5, v_6)\}$

A graph is like a road map. Cities are vertices. Roads from city to city are edges

An Example of a Graph



G = (V, E)

Where, V - set of vertices

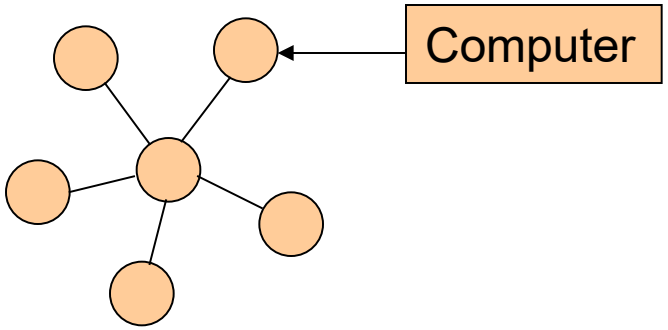
E - set of edges

V = {1, 2, 3, 4, 5}

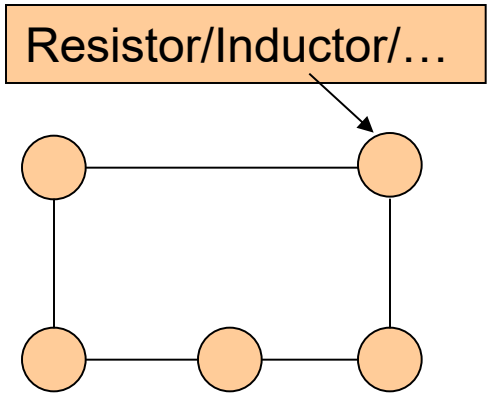
E = { (1,2), (1,3), (1,4), (2,3), (3,5), (4,5) }

Applications of Graphs

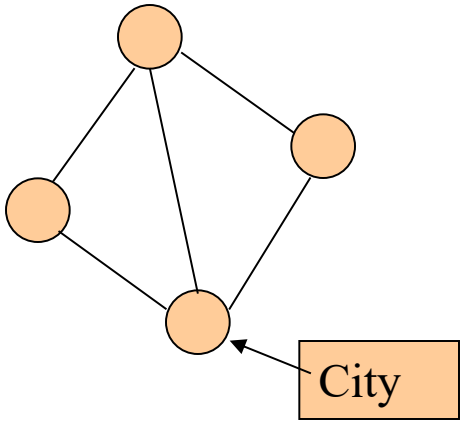
- To model computer networks



- To model electrical circuits



- To model road networks



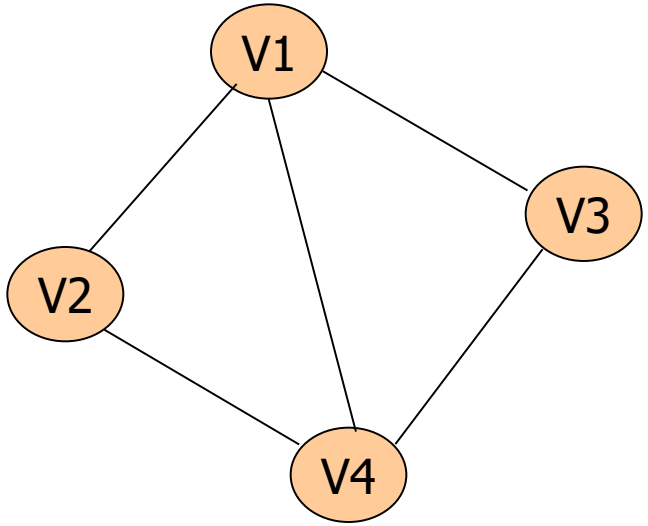
Different Types of Graphs

- Simple/undirected graph
- Directed graph
- Multi graph

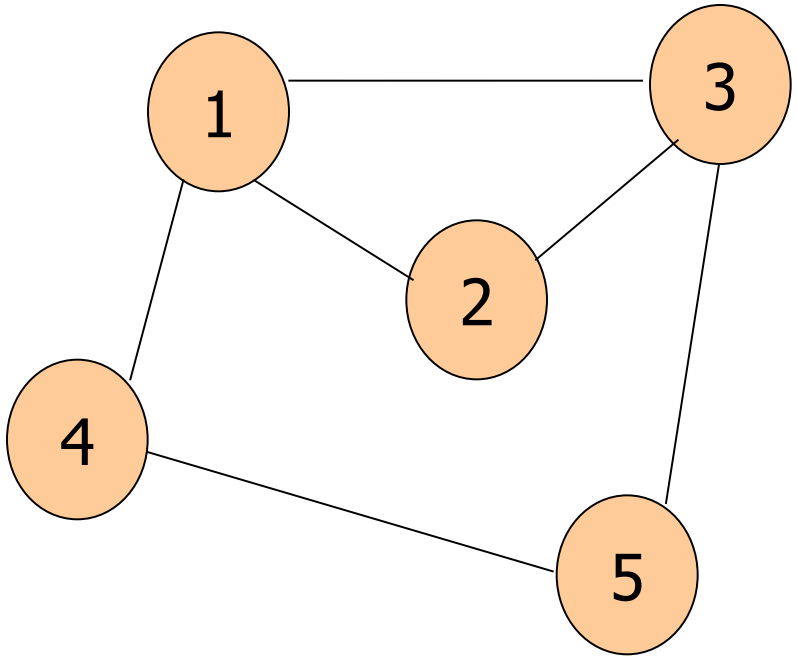
Undirected Graph

- An *Undirected Graph* is a graph where the edges have no directions.
- The edges in an undirected graph are called *Undirected Edges*.

$$\{vi,vj\} = \{vj, vi\}$$



- Example (Undirected Graph)



G = (V, E)

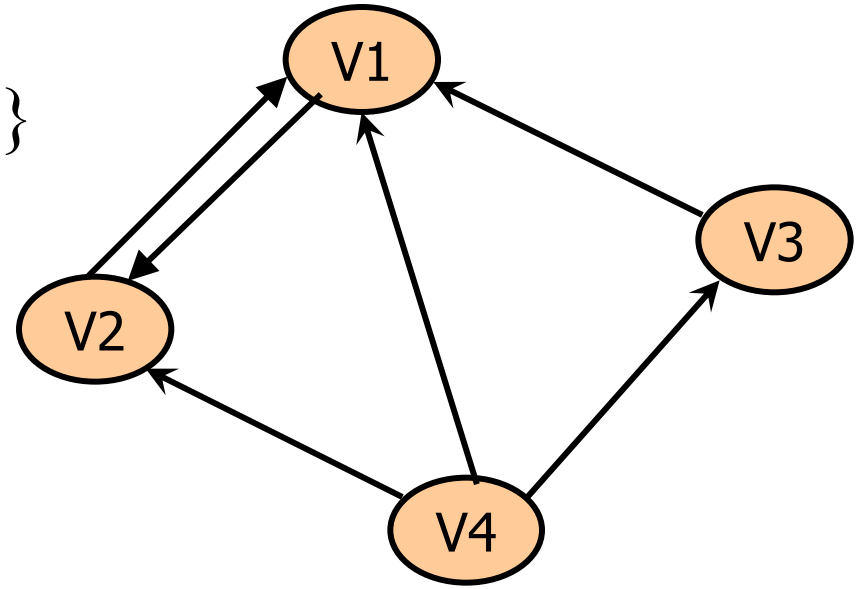
V = {1, 2, 3, 4, 5}

E = {(1,2), (1,3), (1,4), (2,3), (3,5), (4,5)}

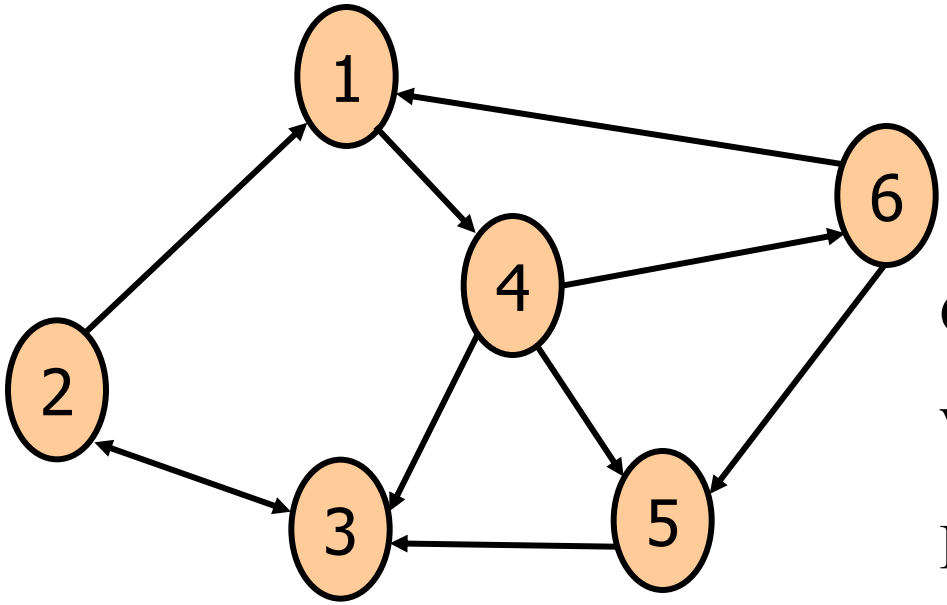
Directed Graphs

- A *Directed Graph* or *Digraph* is a graph where each edge has a direction.
- The edges in a digraph are called *Arcs* or *Directed Edges*.

$\{vi,vj\} \neq \{vj, vi\}$



- Example (Digraph)



G = (V, E)

V = {1, 2, 3, 4, 5, 6}

E = {(1,4), (2,1), (2,3), (3,2), (4,3), (4,5), (4,6), (5,3), (6,1), (6,5)}

(1, 4) = 1→4 where 1 is the *tail*

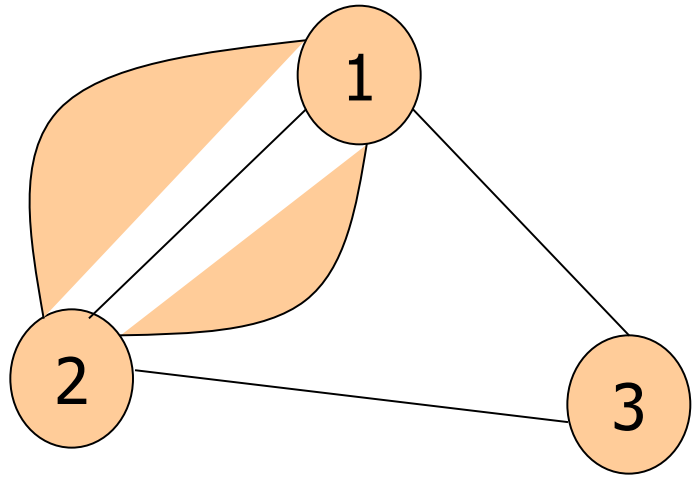
and 4 is the *head*

Multi Graph

- If two vertices can be joined by multiple edges, that graph is called a *Multi graph*.

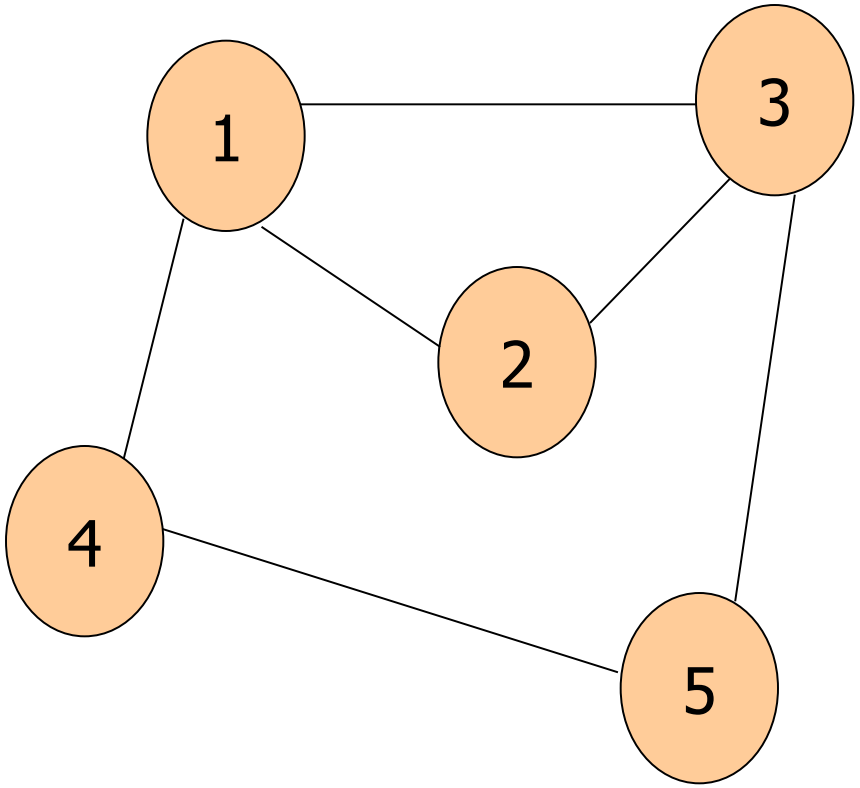
$E \longrightarrow \{Vi, Vj\}$

Where $Vi \neq Vj$



Graph Terminology

- *Adjacent vertices:* If (i,j) is an edge of the graph, then the nodes i and j are adjacent
- An edge (i,j) is *Incident* to *vertices i and j*



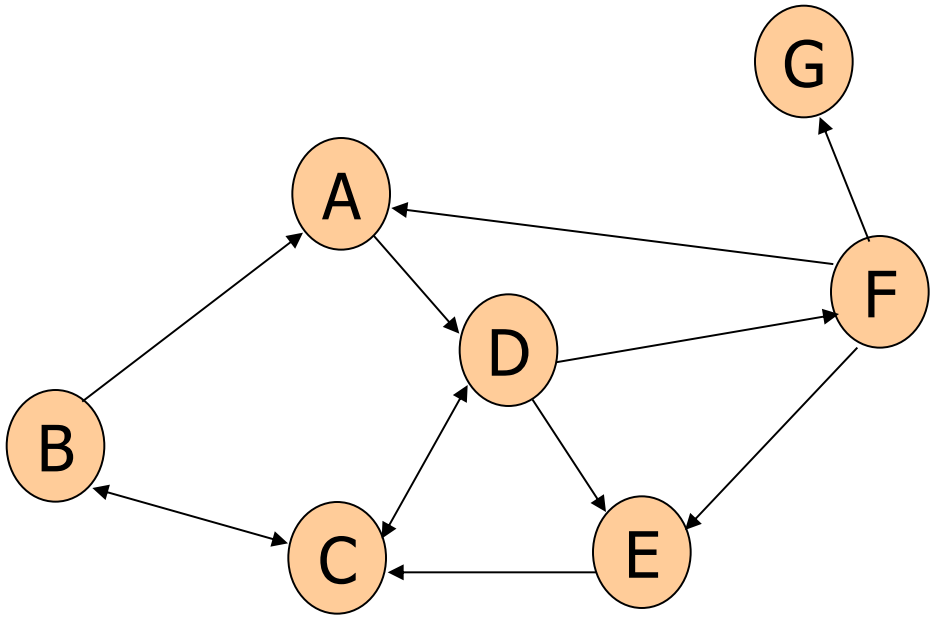
Vertices 2 and 5 are *not* adjacent

Graph Terminology

- *Path:* A sequence of edges in the graph
- There can be more than one path between two vertices

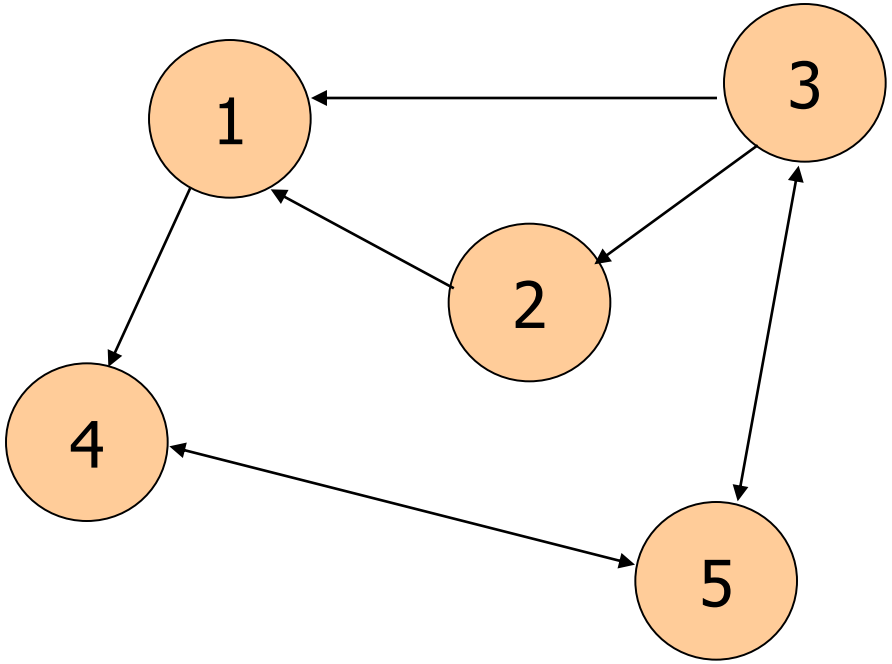
  Paths from B to D :
  B, A, D and B, C, D

- Vertex A is *reachable* from B if there is a path from A to B



Graph Terminology

- *Simple Path:* A path where all the vertices are distinct

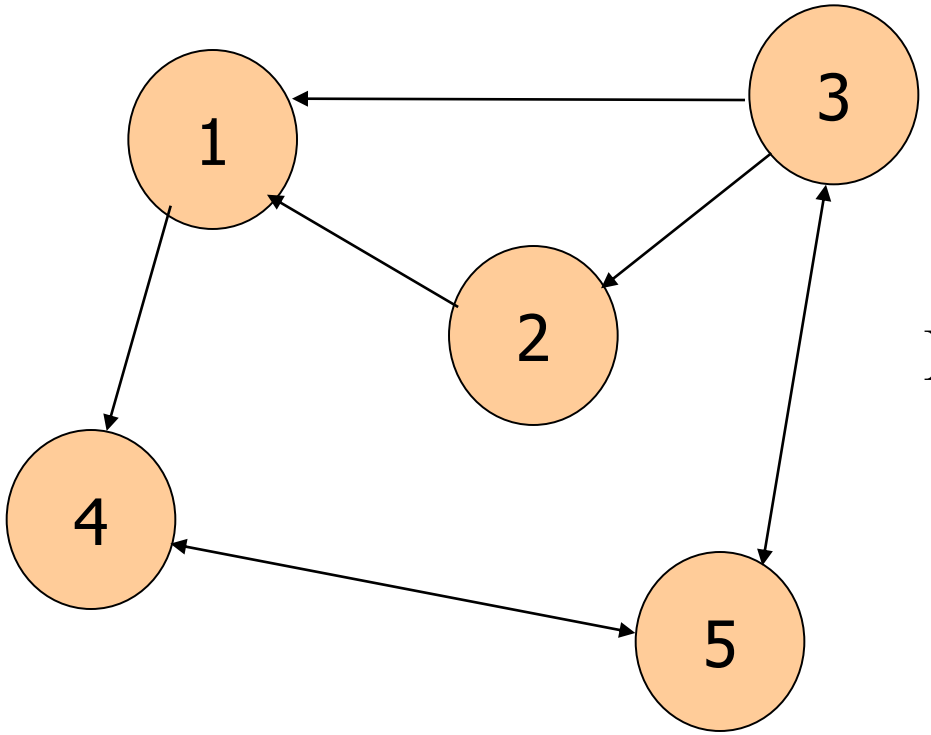


1,4,5,3 is a simple path.

But 1,4,5,4 is not a simple path.

Graph Terminology

- *Length :* Sum of the lengths of the edges on the path.



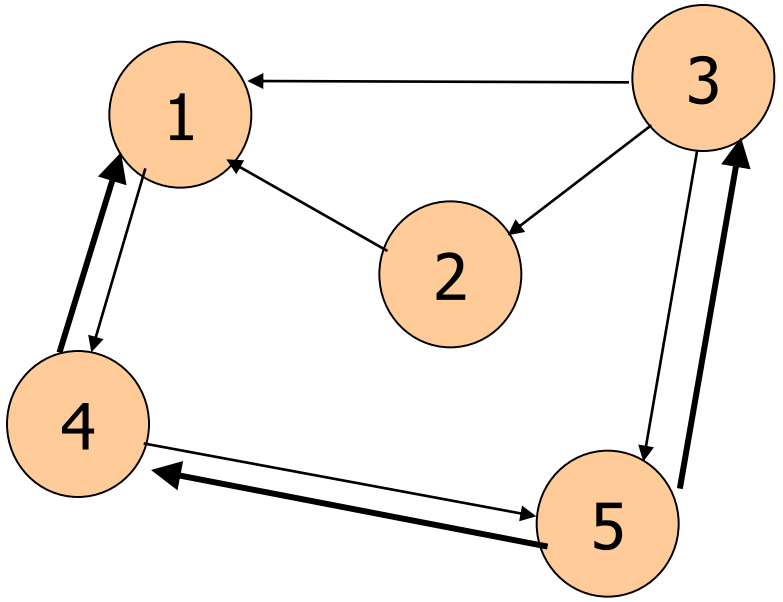
Length of the path 1,4,5,3 is 3

Graph Terminology

- *Degree of a Vertex :* In an undirected graph, the no. of edges incident to the vertex
- *In-degree:* The no. of edges entering the vertex in a digraph
- *Out-Degree:* The no. of edges leaving the vertex in a digraph

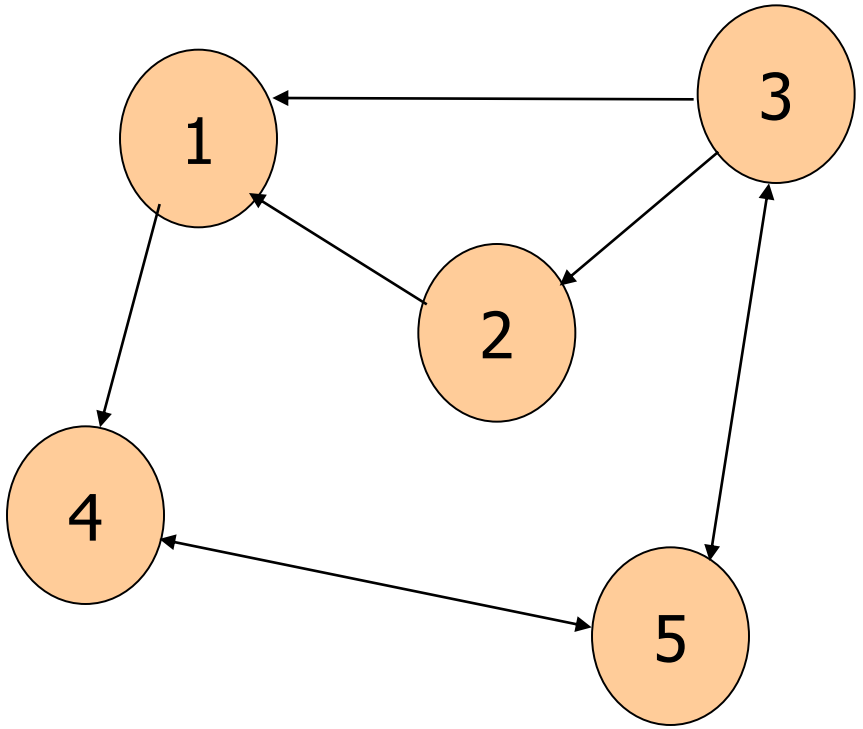
In-degree of 1 is 3

Out-degree of 1 is 1



Graph Terminology

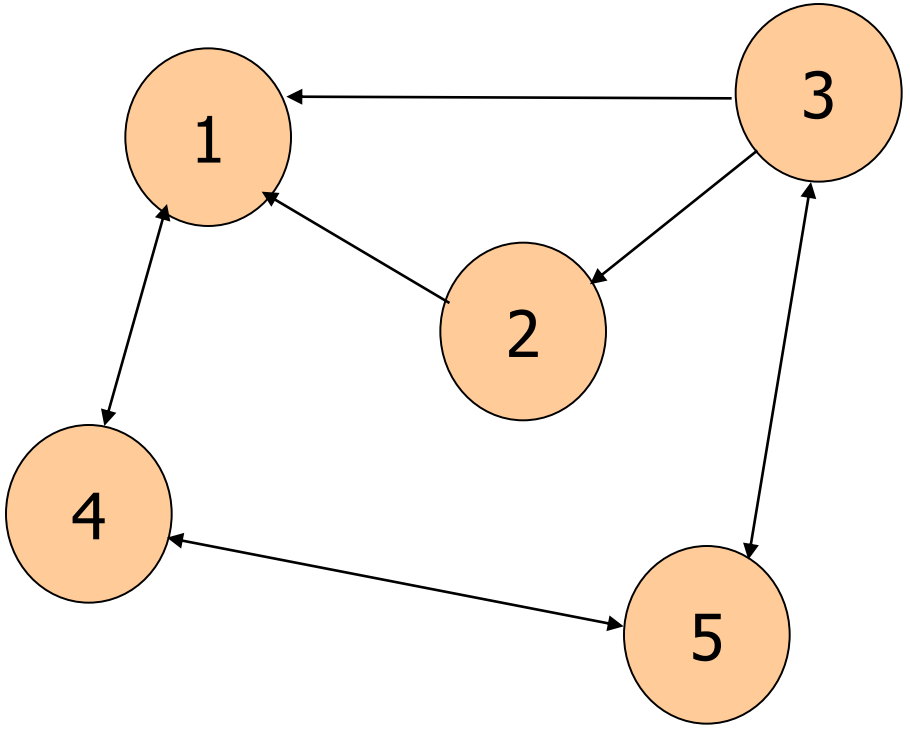
- *Circuit:* A path whose first and last vertices are the same and no edge is repeated then the path is called a circuit.



The path 3,2,1,4,5,3 is a circuit.

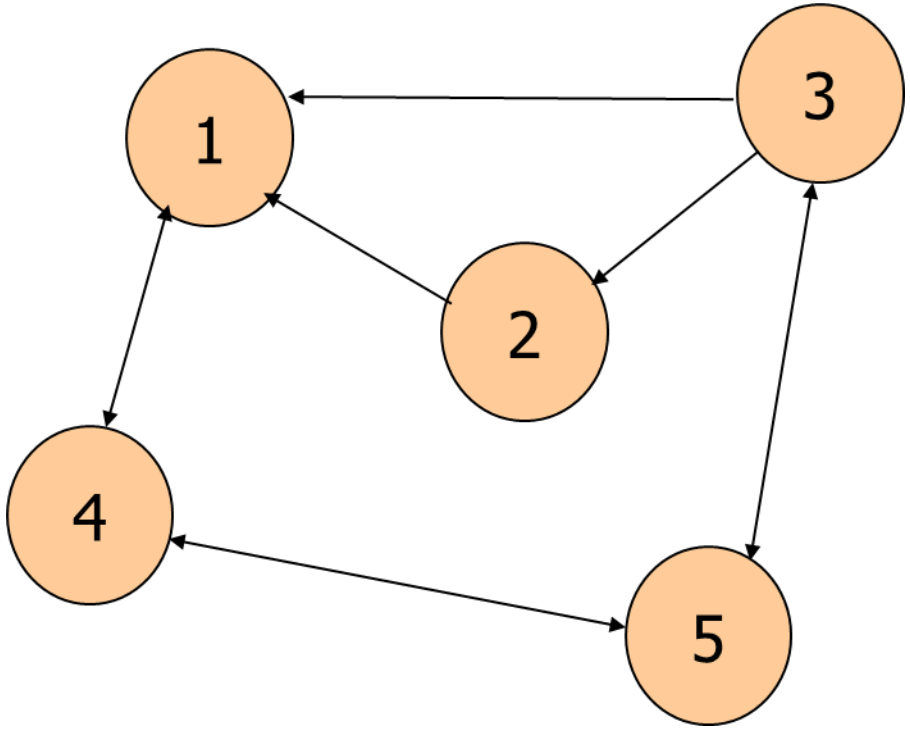
Graph Terminology

- *Cycle:* A circuit where all the vertices are distinct except for the first (and the last) vertex.



1,4,5,3,1 is a cycle

1,4,5,4,1 is not a cycle



Walk : Vertices may repeat. Edges may repeat (Closed or Open)

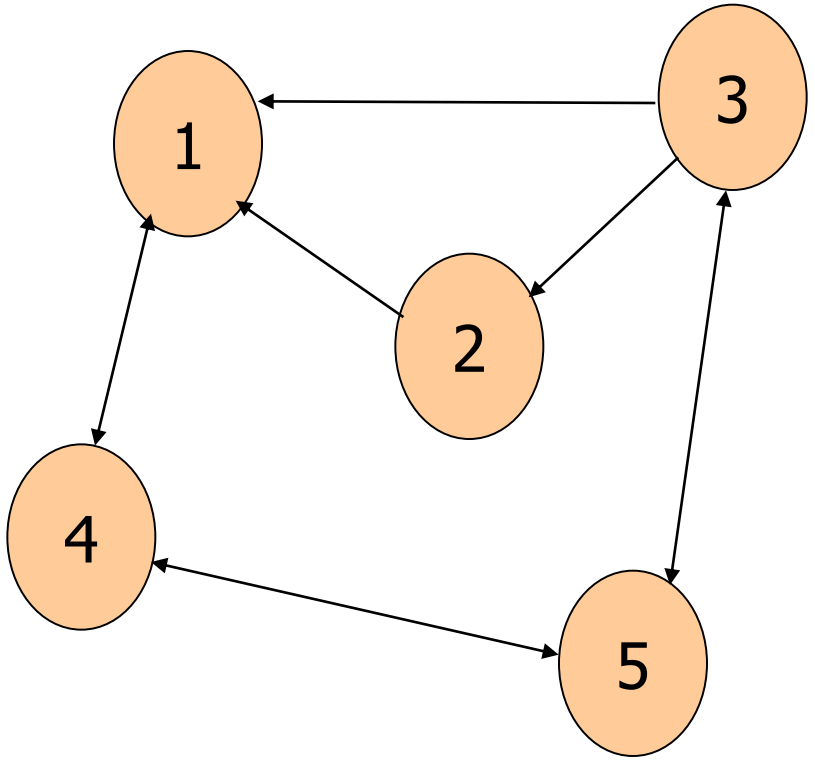
Trail : Vertices may repeat. Edges cannot repeat (Open)

Circuit : Vertices may repeat. Edges cannot repeat (Closed)

Cycle : Vertices cannot repeat. Edges cannot repeat (Closed)

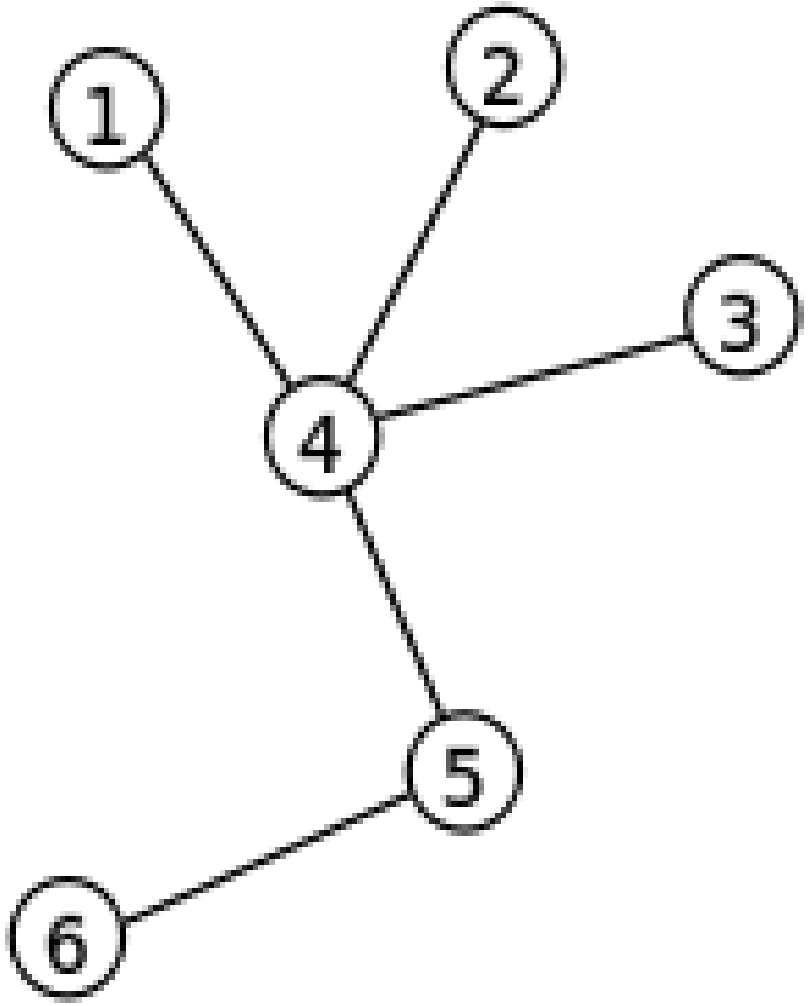
Graph Terminology

- *Hamiltonian Cycle:* A Cycle that contains all the vertices of the graph



1,4,5,3,2,1 is a Hamiltonian Cycle

Tree

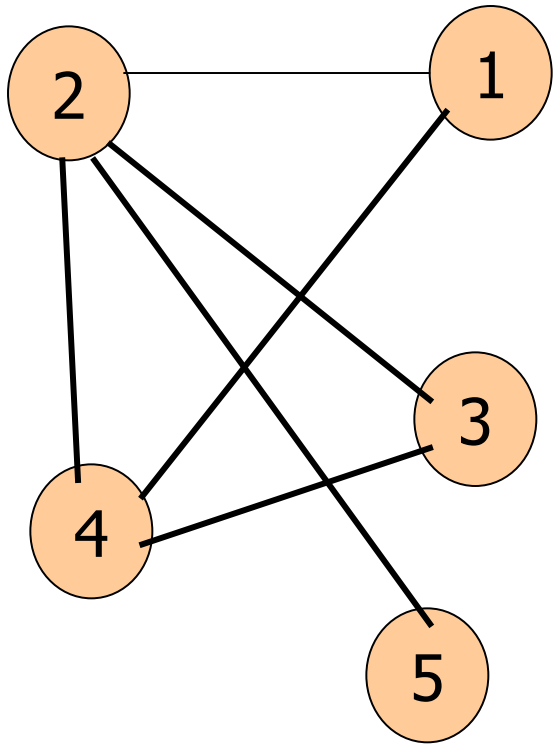


A tree with
6 vertices
and
5 edges

Tree

- Connected and contains no cycles.
- Has V-1 number of edges.
- Any two vertices of a Tree are connected by exactly one path.
- Tree contains no cycles, but the addition of any new edge creates exactly one cycles.

Question: Which of the following sequence forms a cycle/tree?



a. 2,1,4,3,2 ?

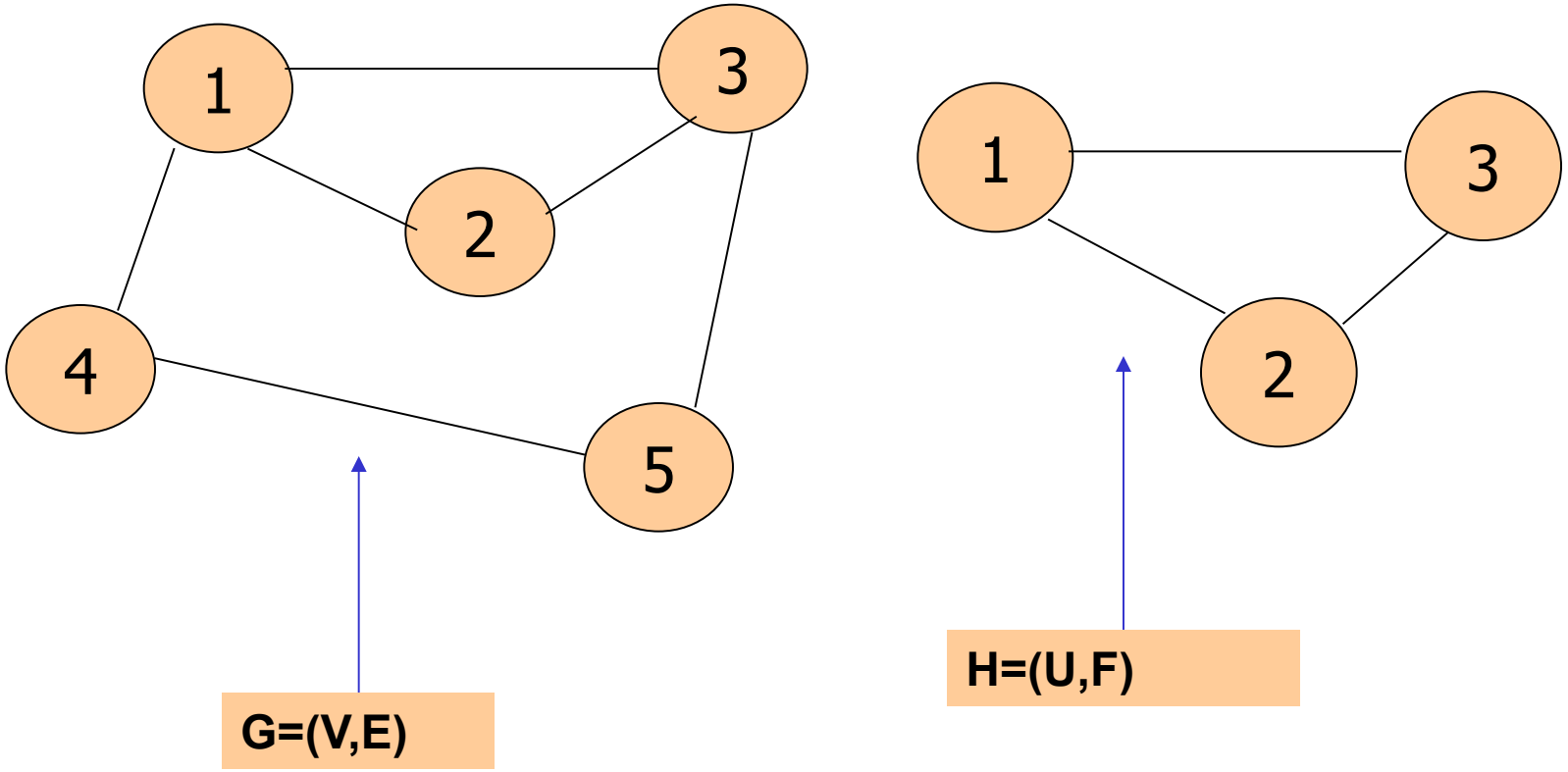
b. 4,1,2,5 ?

Trees & graphs

- Trees can only represent relations of a hierarchical type.
- Graph is a data structure in which this limitation is lifted.
- A graph need not be a tree but a tree must be a graph.

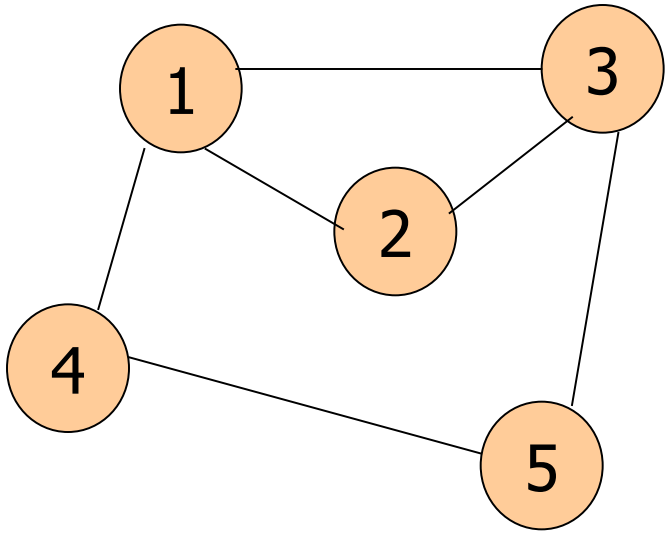
Graph Terminology

- A *Sub graph* of graph G=(V,E) is a graph H=(U,F) such that U Є V and F Є E

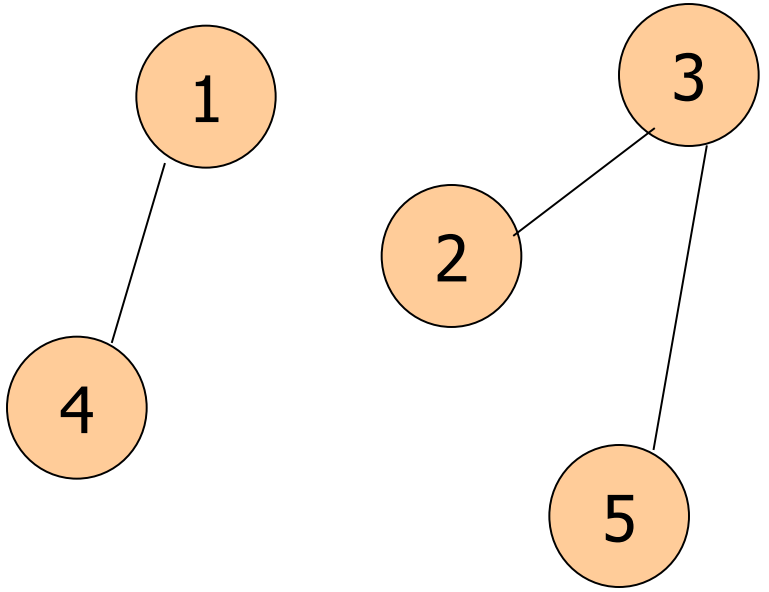


Graph Terminology

- An undirected graph is said to be *Connected* if there is at least one path from every vertex to every other vertex in the graph



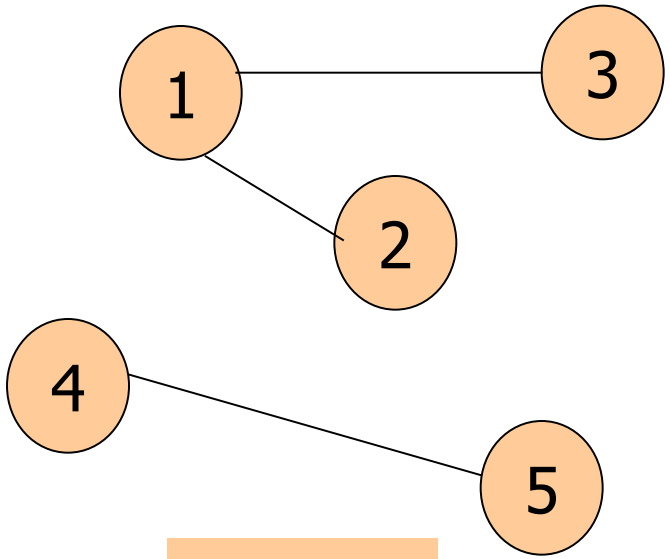
Connected



Unconnected

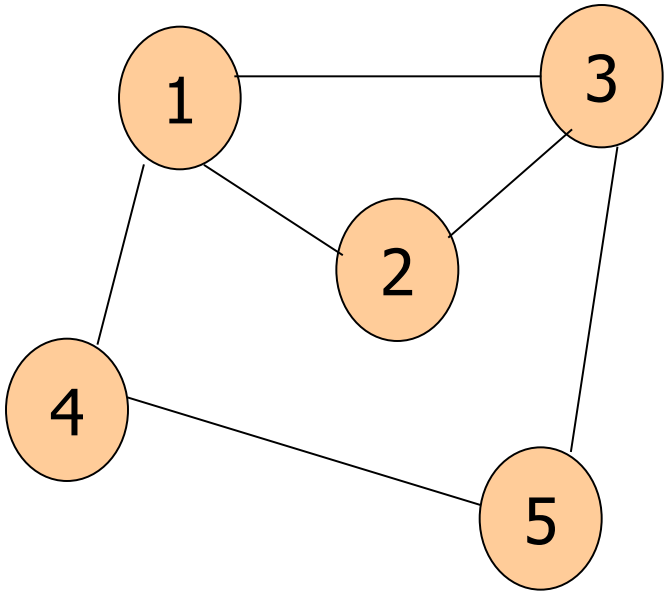
Graph Terminology

- *Forest:* Disjoint union of trees.

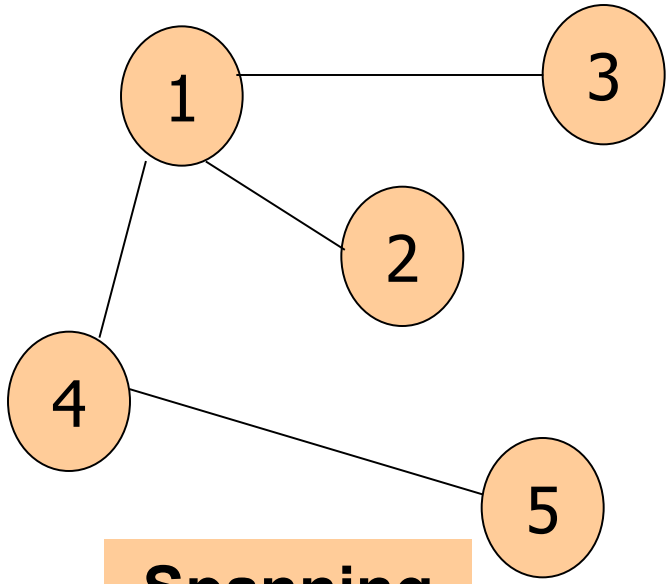


Graph Terminology

- The *Spanning Tree* of a Graph G is a sub graph of G that is a tree and contains all the vertices of G.

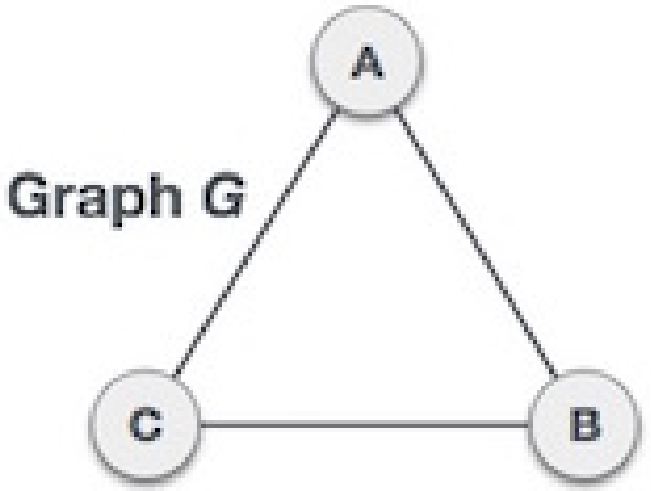


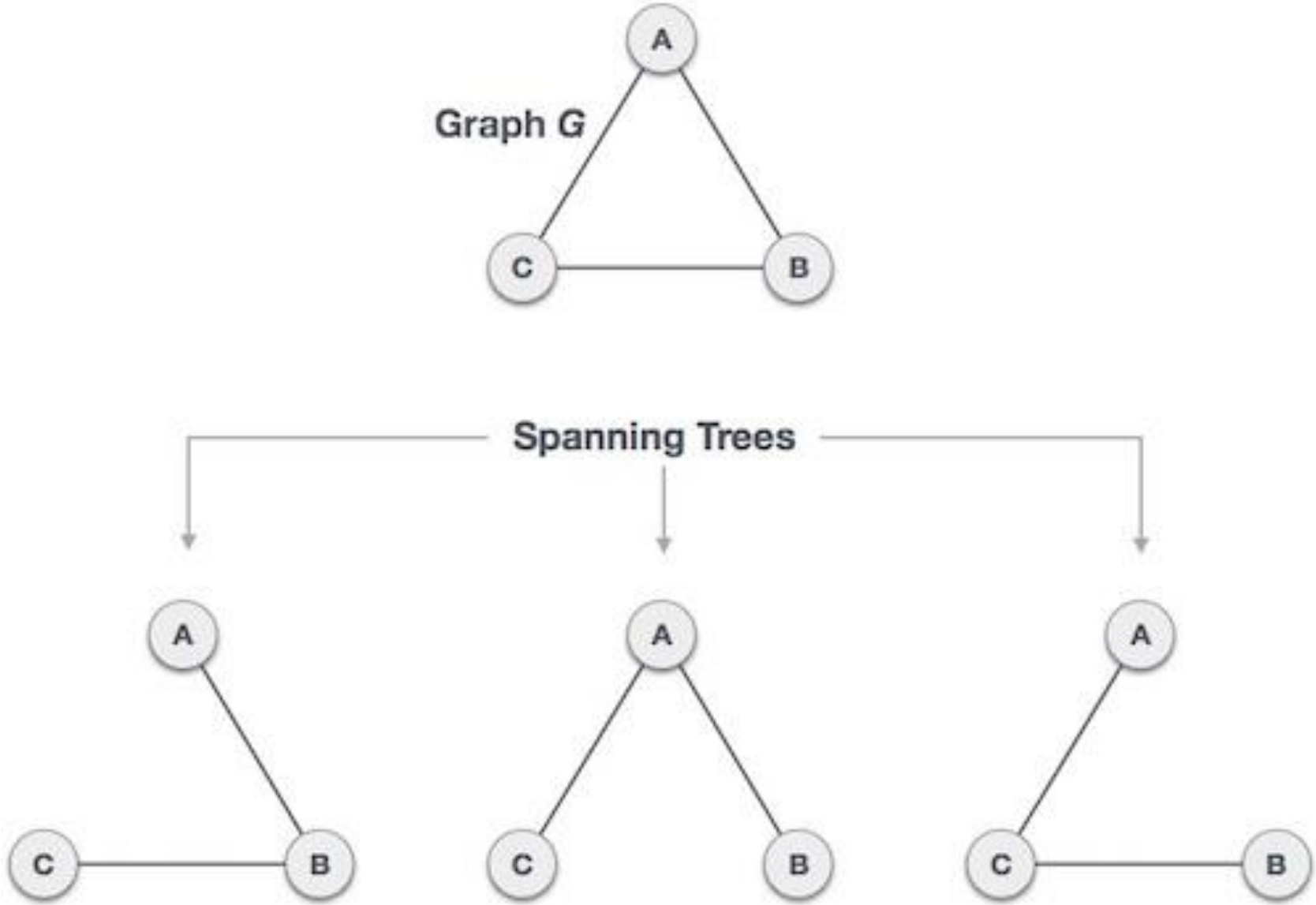
Graph

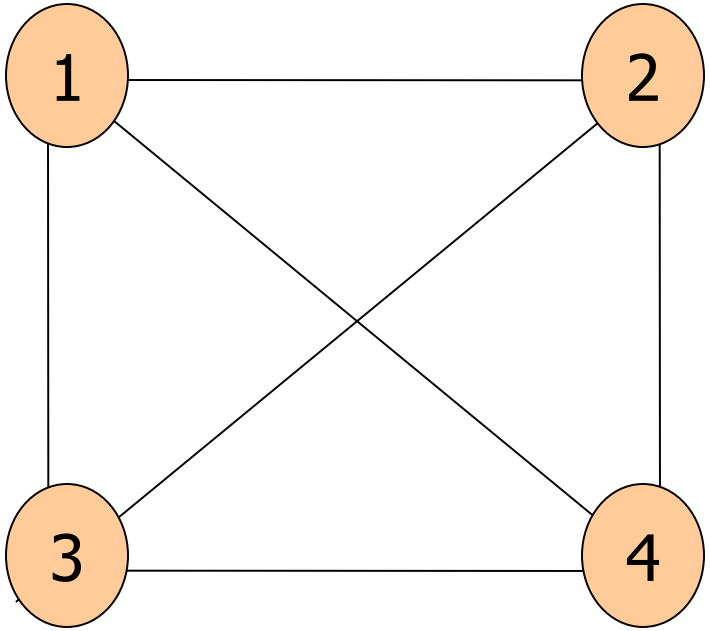


Spanning Tree

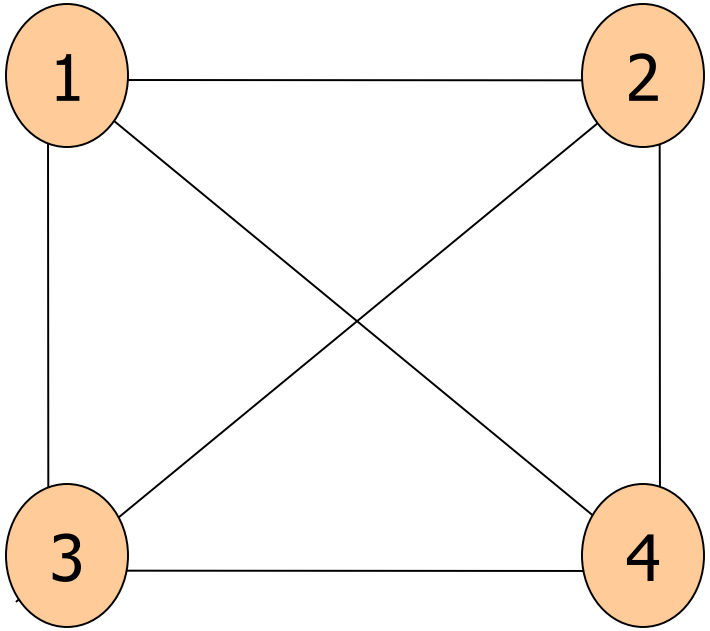
Activity: Draw possible spanning tree(s) for the Graph below




Graph G
A
C
B
Spanning Trees
A
C
B
A
C
B
A
C
B



How many spanning trees can be derived from the above graph?



A complete graph can have maximum $\mathbf{n^{n-2}}$ number of spanning trees.

Representation of Graphs

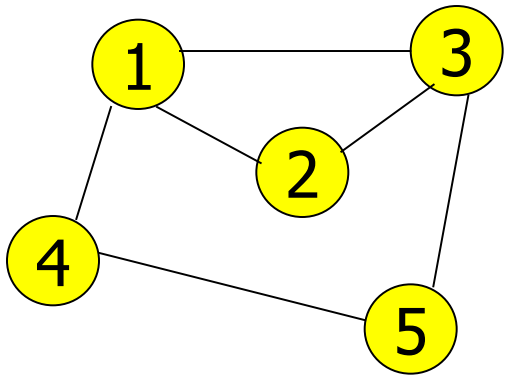
- ***Adjacency Matrix (A)***

  - The Adjacency Matrix $A=(a_{i,j})$ of a graph G=(V,E) with |V| *nodes* is an |V| X |V| matrix
  - Each element of A is either *0 or 1*, depending on the adjacency of the nodes

$$a_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

Representation of Graphs

- *Eg:* The adjacency matrices of the following graphs

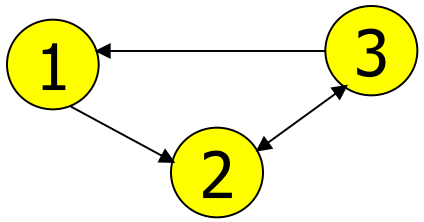


	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	0	0
3	1	1	0	0	1
4	1	0	0	0	1
5	0	0	1	1	0

For undirected graphs, matrix A is symmetric:

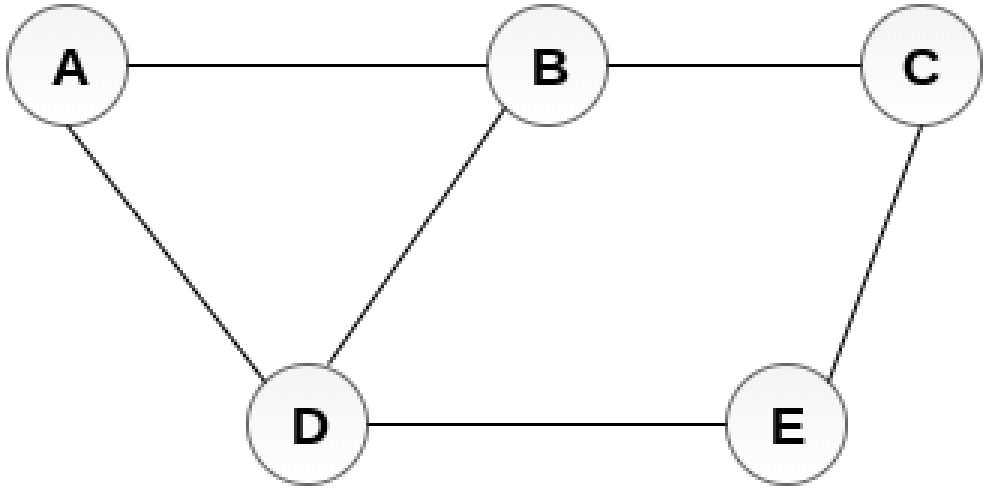
$a_{ij} = a_{ji}$

$A = A^T$

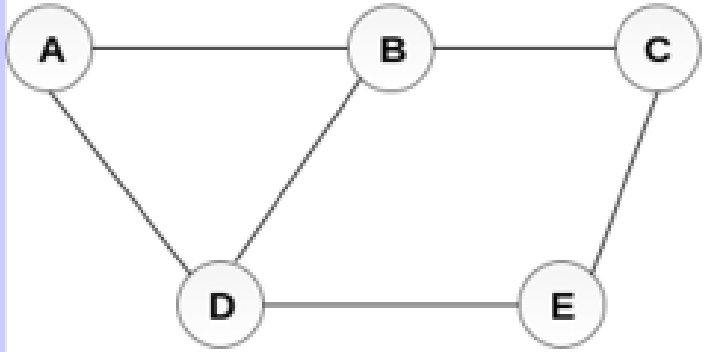


0	1	0
0	0	1
1	1	0

Activity : Give the adjacency matrix for the below graph



Undirected Graph



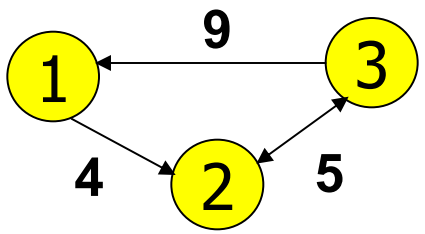
Undirected Graph

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

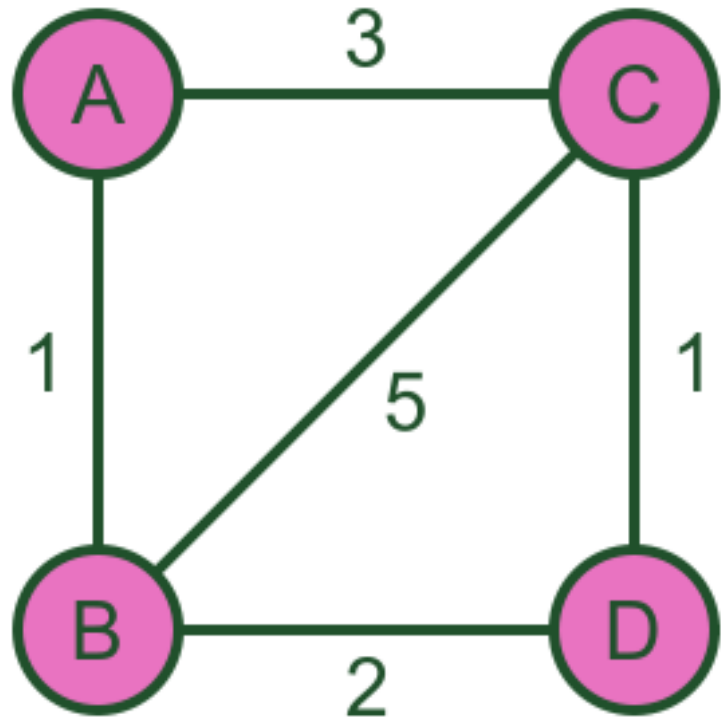
Adjacency Matrix

Representation of Graphs

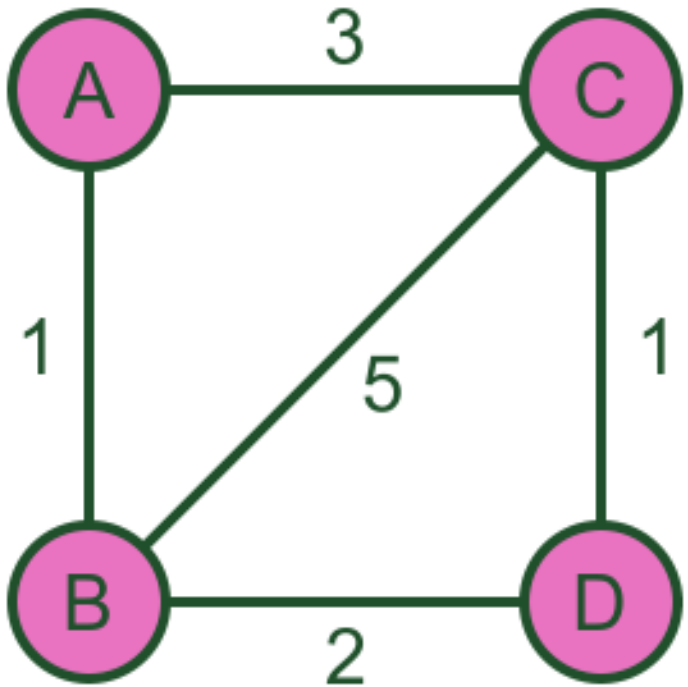
- *Adjacency Matrix* of a *Weighted Graph*
  - The weight of the edge can be shown in the matrix when the vertices are adjacent
  - A nil value (*0 or* ∞) depending on the problem is used when they are not adjacent
  - Eg. To find the minimum distance between nodes...



∞	4	∞
∞	∞	5
9	5	∞



Weighted graph



Weighted graph

	A	B	C	D
A	0	1	3	0
B	1	0	5	2
C	3	5	0	1
D	0	2	1	0

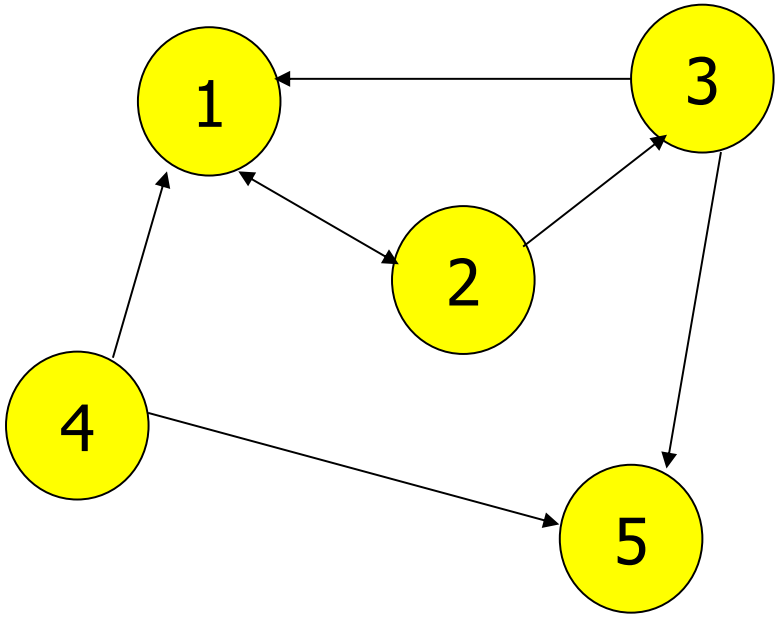
Representation of Graphs

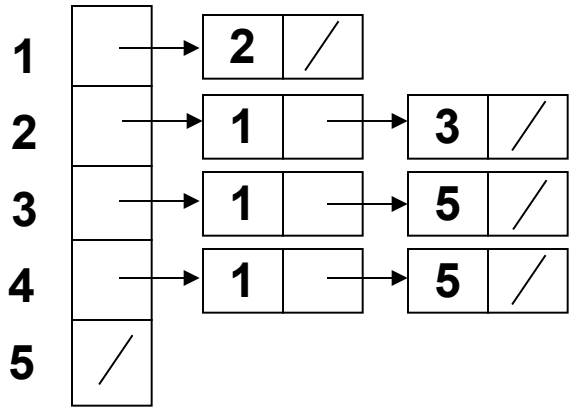
Adjacency matrix representation of a graph can be inadequate since

- It requires advance knowledge of the number of nodes.
- It must be dynamically updated as the program proceeds and a new matrix must be created for each addition or deletion of a node.
- Space must be reserved for every possible edge between two nodes whether or not such an edge exists.

Representation of Graphs

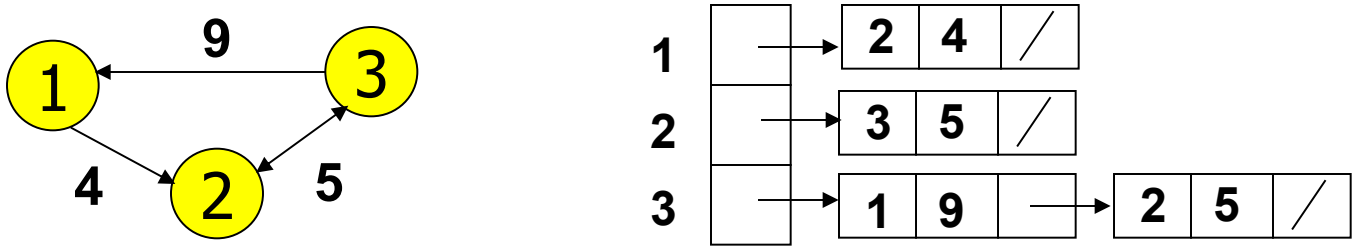
- *Adjacency List*
  - An Adjacency list is an array of lists, each list showing the vertices a given vertex is adjacent to



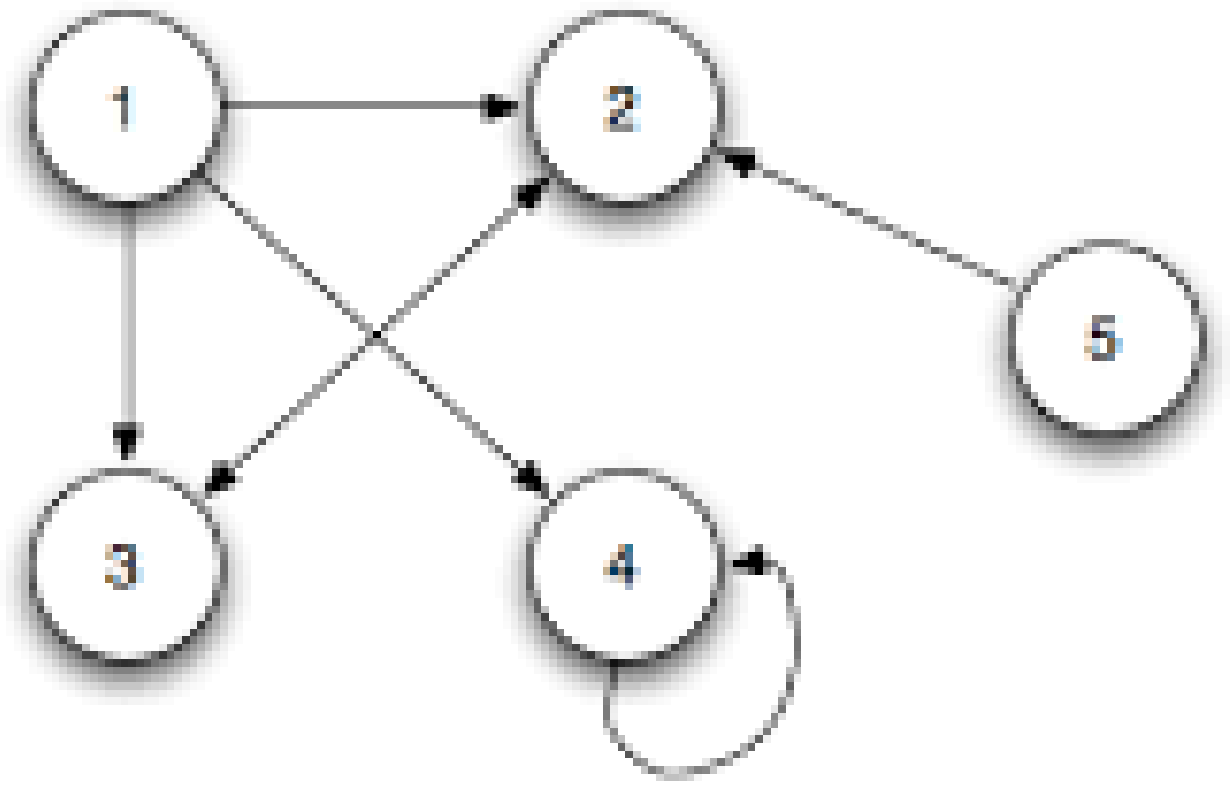


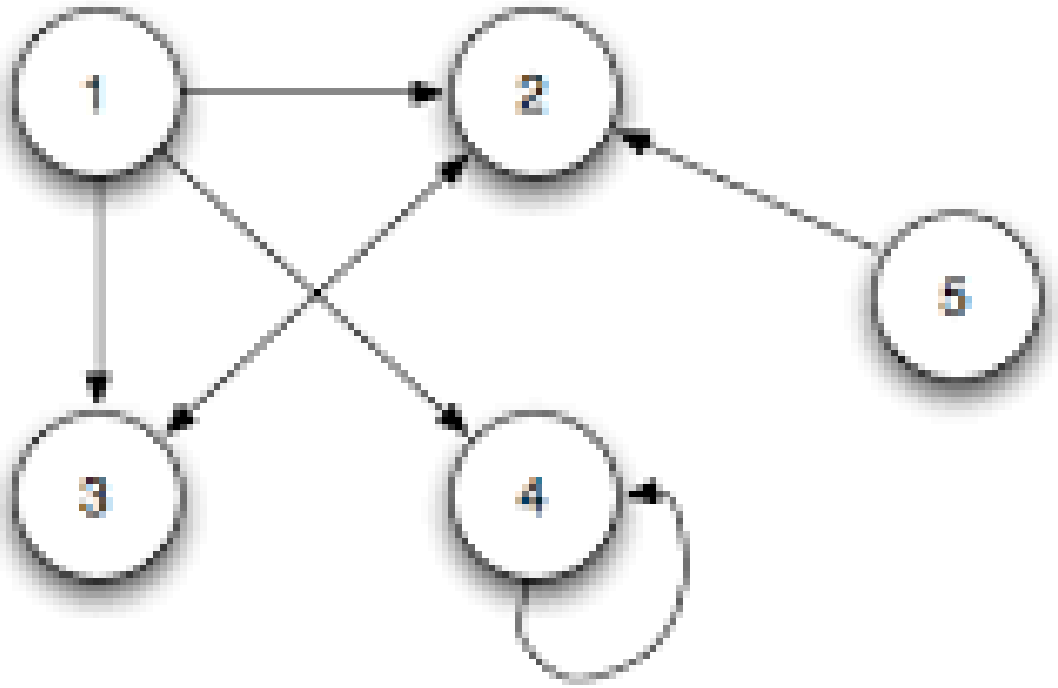
Representation of Graphs

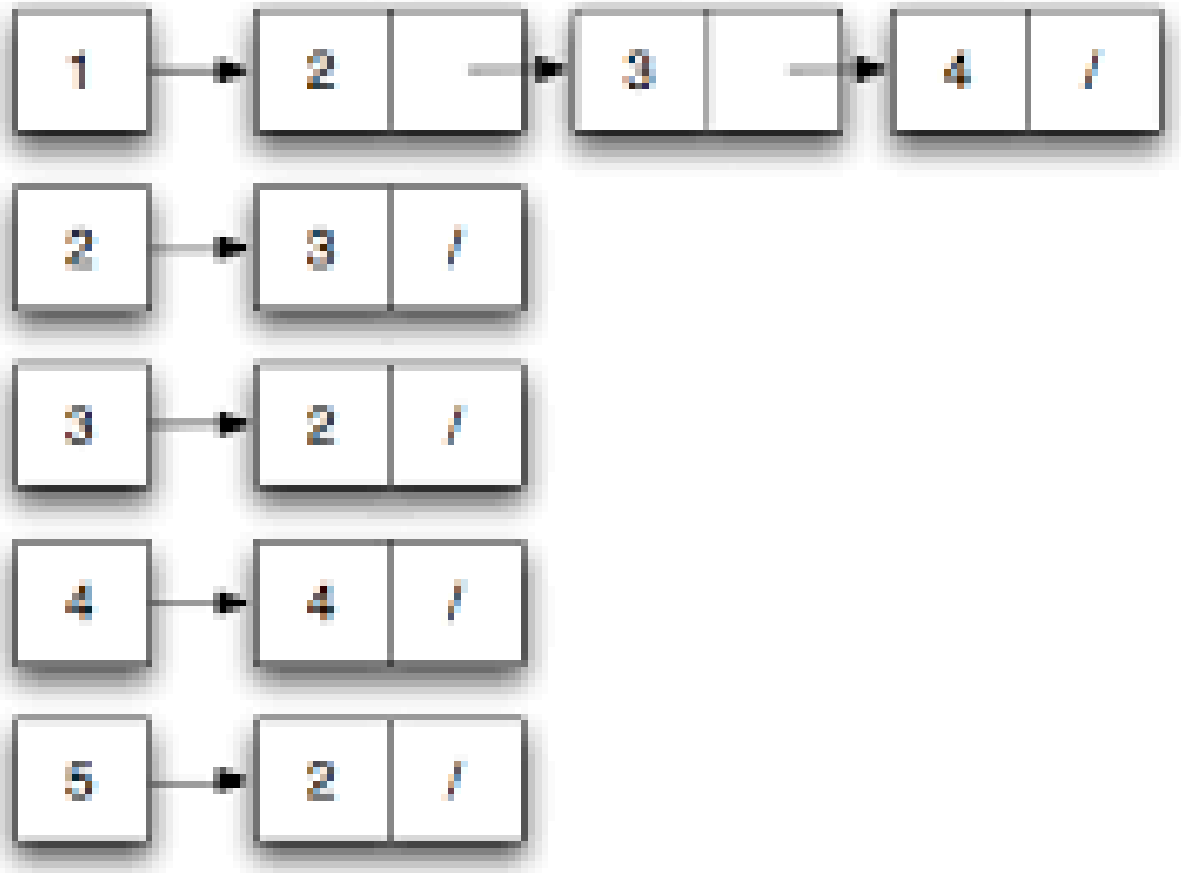
- *Adjacency List* of a *Weighted Graph*
  - The weight is included in the list



Activity: Draw the adjacency list for the below graph




1
2
3
4
5

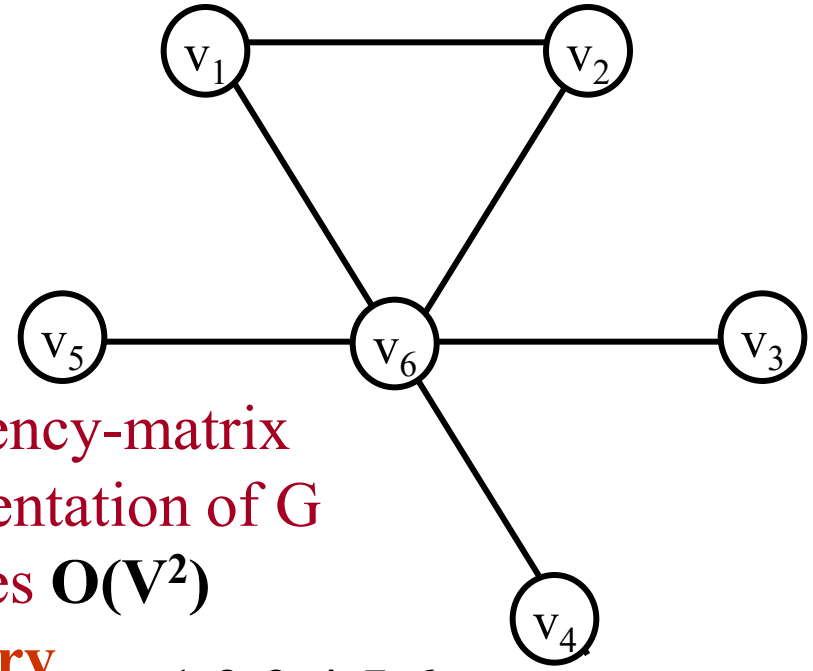

1
2
3
4
/
2
3
/
3
2
/
4
4
/
5
2
/

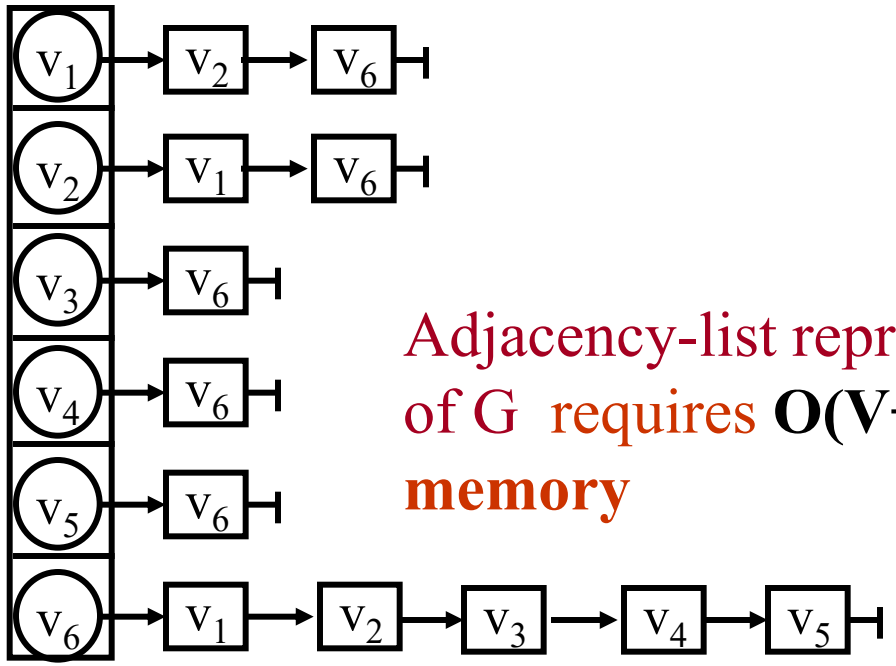
Undirected Graph (data structure)

$G = (V,E)$

$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$

$E = \{(v_1, v_2), (v_1, v_6), (v_2, v_6), (v_3, v_6), (v_4, v_6), (v_5, v_6)\}$





Adjacency-list representation of G requires **O(V+E)** **memory**

Adjacency-matrix representation of G requires **O(V^2)** **memory**

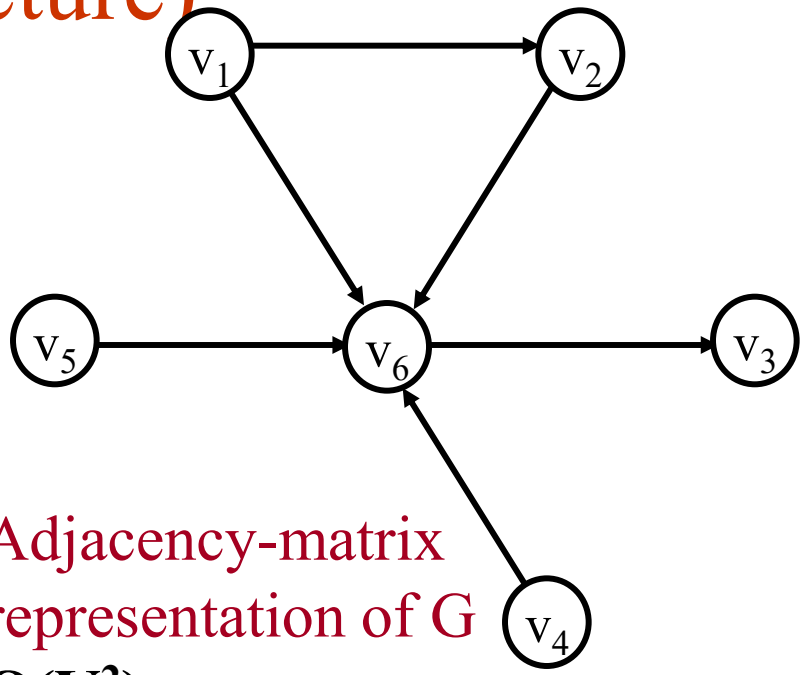
	1	**2**	**3**	**4**	**5**	**6**
1	0	1	0	0	0	1
2	1	0	0	0	0	1
3	0	0	0	0	0	1
4	0	0	0	0	0	1
5	0	0	0	0	0	1
6	1	1	1	1	1	0

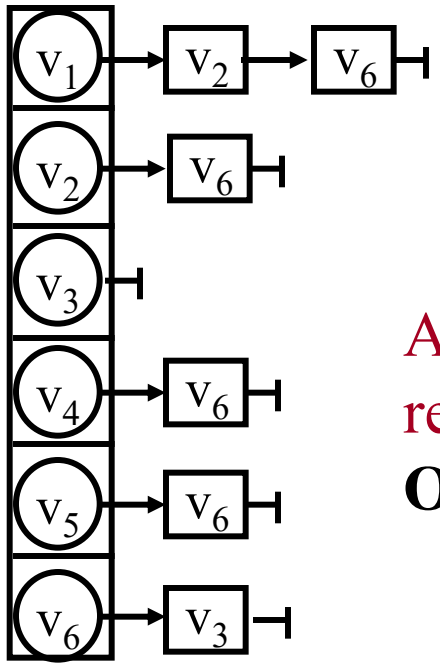
Directed Graph (data structure)

$G = (V,E)$

$V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$

$E = \{\langle v_1, v_2\rangle, \langle v_1, v_6\rangle, \langle v_2, v_6\rangle, \langle v_6, v_3\rangle, \langle v_4, v_6\rangle, \langle v_5, v_6\rangle\}$





Adjacency-list representation of G

O(V+E)

Adjacency-matrix representation of G

O(V^2)

	1	**2**	**3**	**4**	**5**	**6**
1	0	1	0	0	0	1
2	0	0	0	0	0	1
3	0	0	0	0	0	0
4	0	0	0	0	0	1
5	0	0	0	0	0	1
6	0	0	1	0	0	0

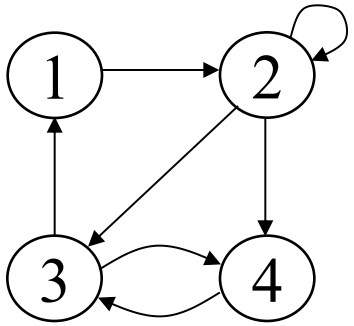
Properties of Adjacency-List Representation

- Sum of "lengths" of all adjacency lists
  - Directed graph: $|E|$

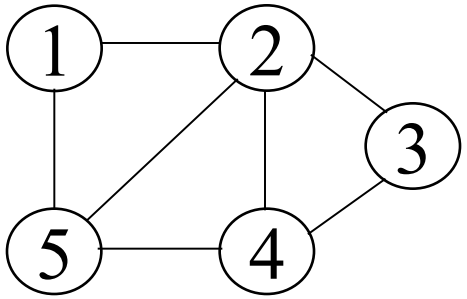
    edge (u, v) appears only once (i.e., in the list of **u**)

  - Undirected graph: $2|E|$

    edge (u, v) appears twice (i.e., in the lists of both **u** and **v**)



Directed graph



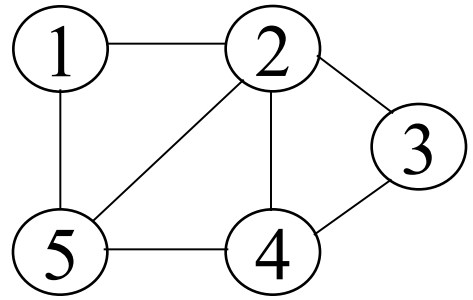
Undirected graph

Properties of Adjacency-List Representation

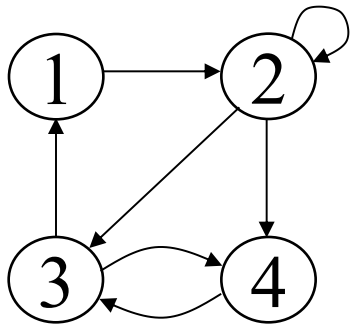
- Memory required

  $\Theta(V + E)$

- Preferred when
  - The graph is **sparse**: $|E| << |V|^2$
  - We need to quickly determine the nodes adjacent to a given node.



Undirected graph



Directed graph

Properties of Adjacency-List Representation

- Disadvantage
  - No quick way to determine whether there is an edge between node u and v
- Time to determine if $(u, v) \in E$:

  O(degree(u))
- Time to list all vertices adjacent to u:

  O(degree(u))

Properties of Adjacency Matrix Representation

- Memory required

  $O(V^2)$, independent on the number of edges in G

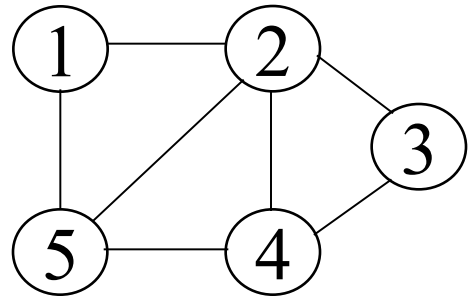
- Preferred when the graph is **dense:**

  $|E|$ is close to $|V|^2$

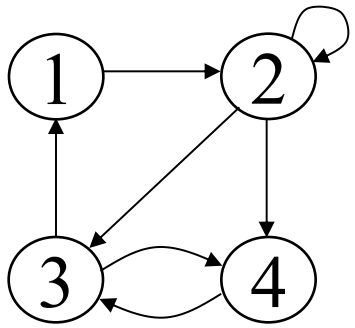
  – We need to quickly determine if there is an edge between two vertices

- Time to determine if $(u, v) \in E$:

  O(1)



Undirected graph



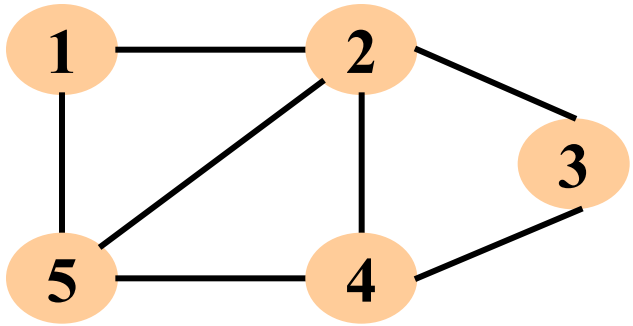
Directed graph

Properties of Adjacency Matrix Representation

- Disadvantage
  - No quick way to determine the vertices adjacent to another vertex
- Time to list all vertices adjacent to u:

  O(V)

Examples

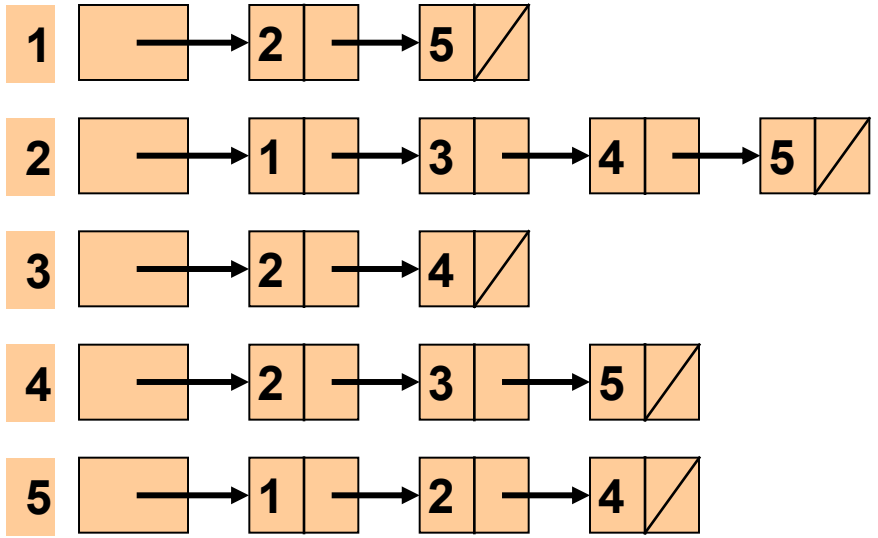


Adjacency Lists

Adjacency Matrices

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

$O(|V|^2)$



$O(|V|+|E|)$

Homework

Draw the graph represented by the following adjacency matrix.

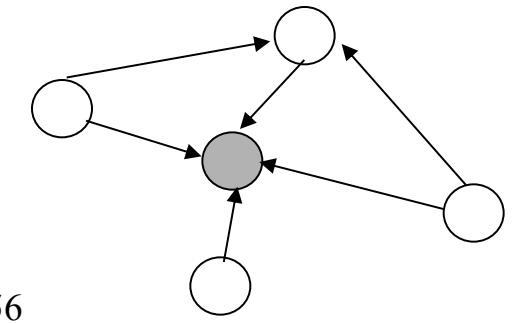
	a	**b**	**c**	**d**	**e**	**f**	**g**
a	0	1	1	0	0	0	0
b	0	0	1	1	0	0	0
c	0	0	0	1	0	0	0
d	0	1	0	0	1	1	1
e	0	1	0	0	0	0	0
f	0	0	1	1	0	0	1
g	0	0	0	0	1	0	0

Draw the adjacency list representation for this graph.

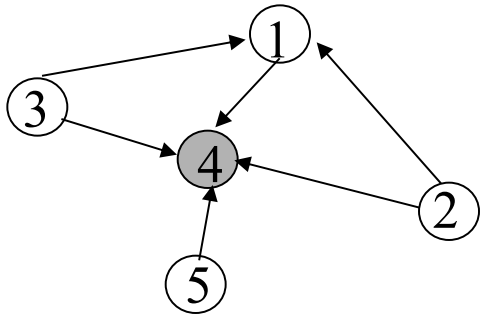
Universal Sink

When **adjacency-matrix representation is used**, most graph algorithms require time $\Omega(V^2)$, but there are some exceptions. Show that determining whether a directed graph G contains a **universal sink** – a vertex of in-degree |V|-1 and out-degree 0 – an be determined in time O(V).

Example



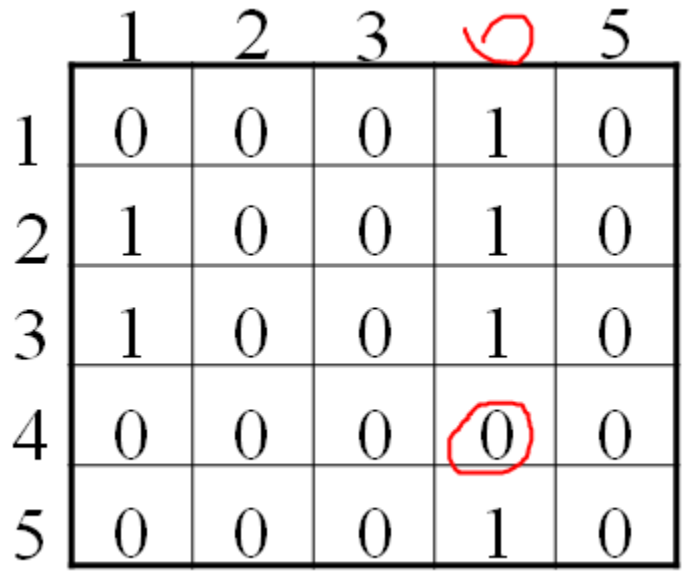
Universal Sink



	1	2	3	4	5
1	0	0	0	1	0
2	1	0	0	1	0
3	1	0	0	1	0
4	0	0	0	0	0
5	0	0	0	1	0

Universal Sink

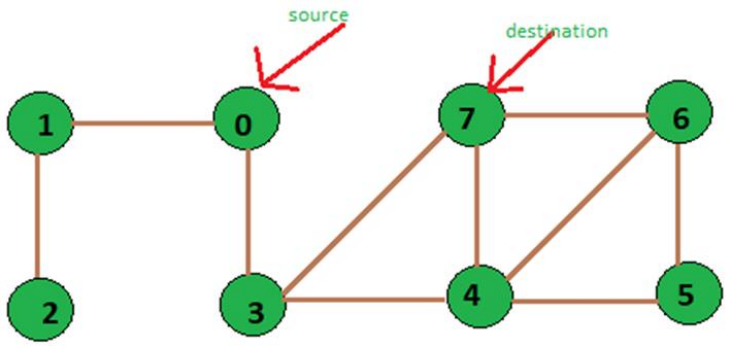
```
Sink (A,k)
    A → v x v
    for j ← 1 to v do
        if A[k,j] == 1
            return false;
    for i ← 1 to v do
        if A[i,k] == 0  and i ≠ k
            return false;
    return true;
```



	1	2	3	4	5
1	0	0	0	1	0
2	1	0	0	1	0
3	1	0	0	1	0
4	0	0	0	0	0
5	0	0	0	1	0

Algorithms for Searching Graphs

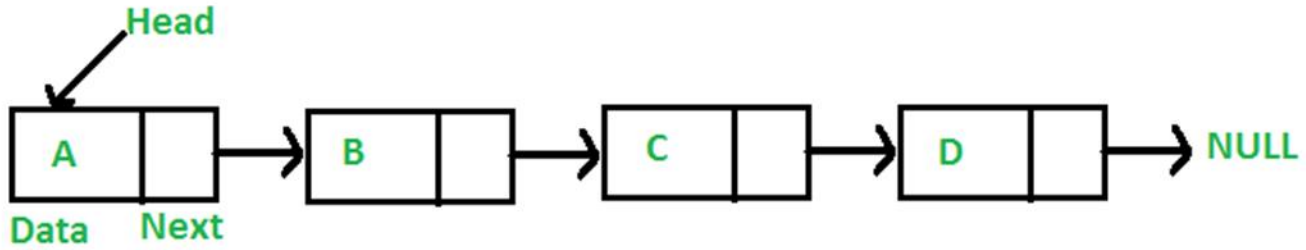
- Why do we need to search graphs
  - To look for connectivity
  - To find paths

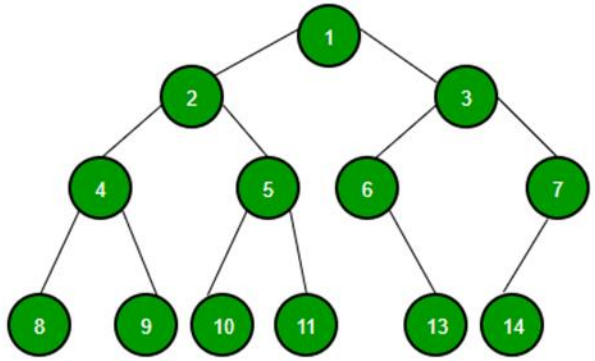


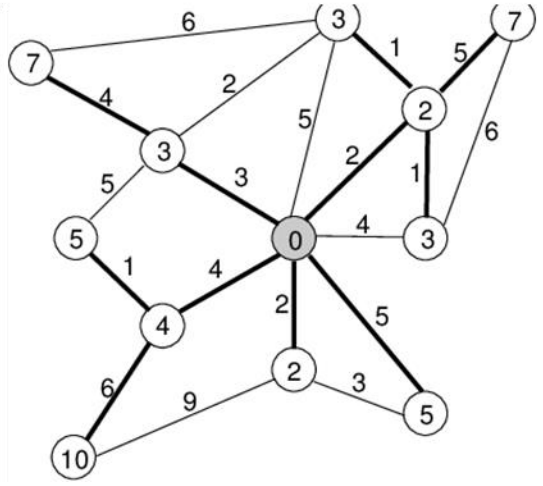
- Two strategies
  - Depth-First Search (DFS)
  - Breadth-First Search (BFS)

Graph Traversal

- Defining graph traversal is more complex than is that for a list or a tree.







Graph Traversal

- No natural ‘first’ node in a graph from which the traversal should start.

- Once a starting node has been determined and all nodes reachable from that node have been visited, there may remain other nodes in the graph that have not been visited since they are not reachable from the starting node.

Graph Traversal

- There is no natural order among the successors of a particular node.

- If a node has more than one predecessor, it will be encountered more than once during a traversal.

Graph Traversal

- Algorithm is either presented with a starting node for the traversal or chooses a random node at which to start.
- Must check that a node being encountered has not been visited previously.
- Implementation of the graph (adjacency list or the matrix) determines the order in which the successors of a node are visited.

Breadth First Search

- Start from an arbitrary node
- Visit all the adjacent nodes (distance = 1)
- Visit the nodes adjacent to the visited nodes (distance=2, 3 etc.)
- Stop when a *dead end* is met
- Implemented using a *Queue*

Explore all nodes at distance d…

Breadth First Search

- BFS calculates the *shortest path distance* to the source node
  - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v, or ∞ if v not reachable from s
- BFS builds *breadth-first tree*, in which paths to root represent shortest paths in G
  - Thus can use BFS to calculate shortest path from one vertex to another in O(V+E) time

Breadth First Search

- Assume using adjacency lists
- Data structures used:

  color[u] = stores color of the vertex

  π[u] = predecessor of u

  If u has no predecessor π[u] = null

  d[u] = discover time array

  Q = Queue of the Vertexes To be Visited

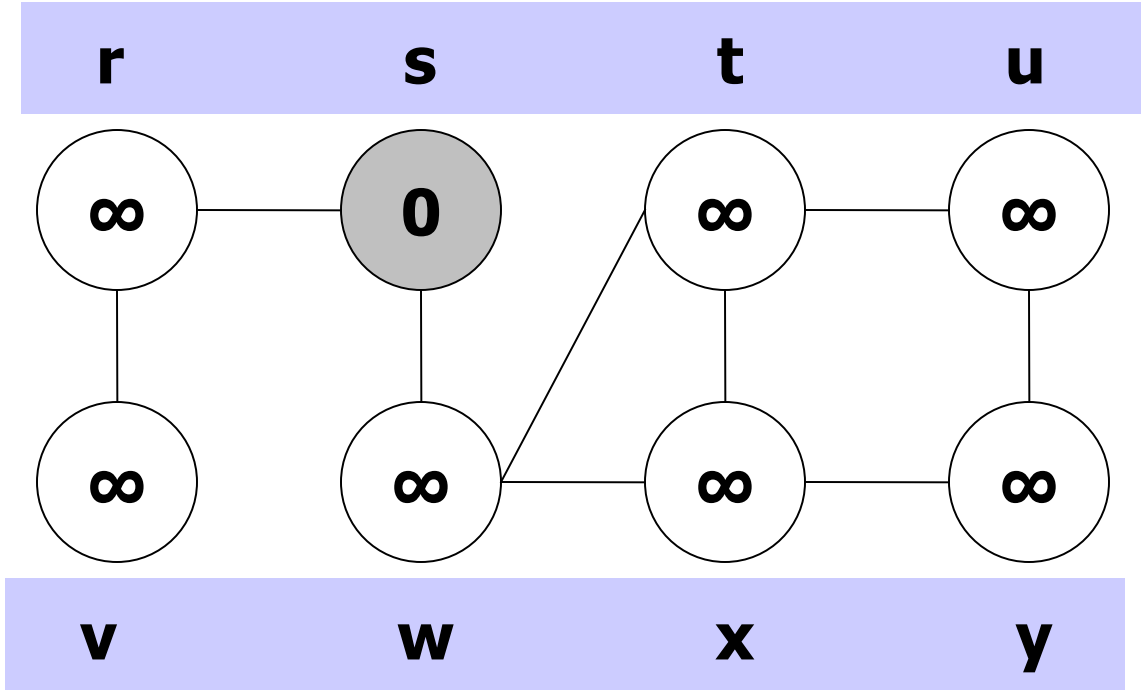
Breadth First Search Algorithm

```
BFS(G,s)
    for each vertex u ∈ V[G] – s
        do color[u] ← WHITE
        d[u] ← ∞
        π[u] ← null
    color[s] ← GRAY
    d[s] ← 0
    π[s] ← null
    Q ← {s}
```

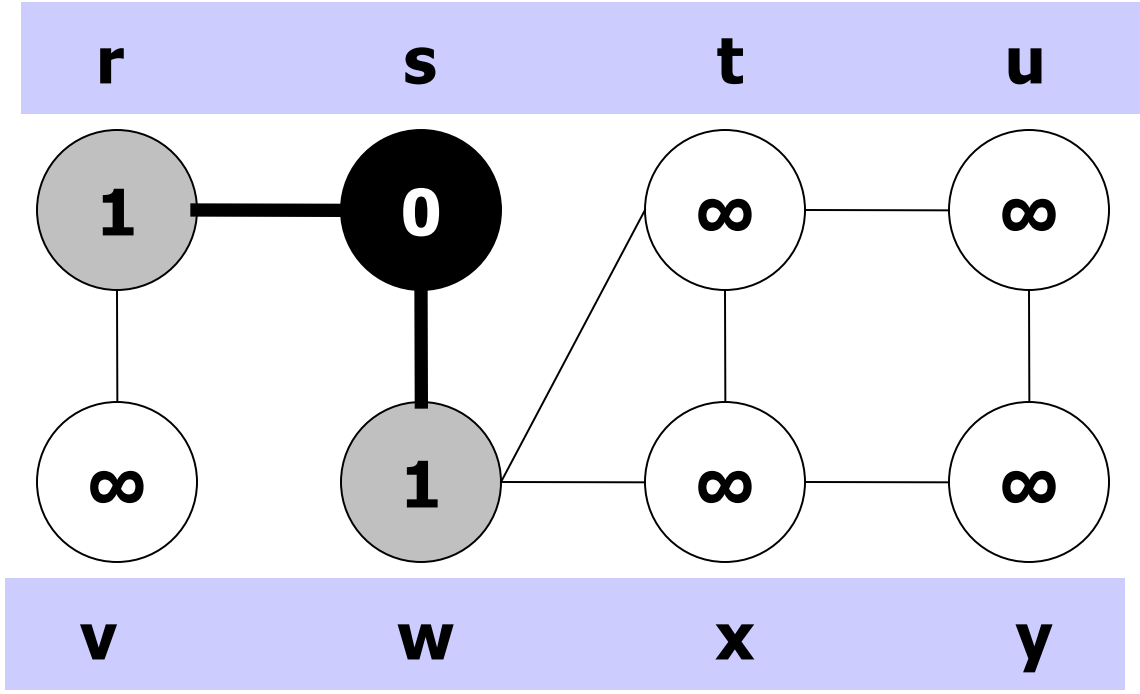
Breadth First Search Algorithm

```
while Q ≠ Ø
        do u ← DEQUEUE(Q)
        for each v ∈ Adj[u]
                do if color[v] = WHITE
                    then color[v] ← GRAY
                         d[v] ←d[u] + 1
                         π[v] ← u
                         ENQUEUE(Q,v)
        color[u] ← BLACK
```

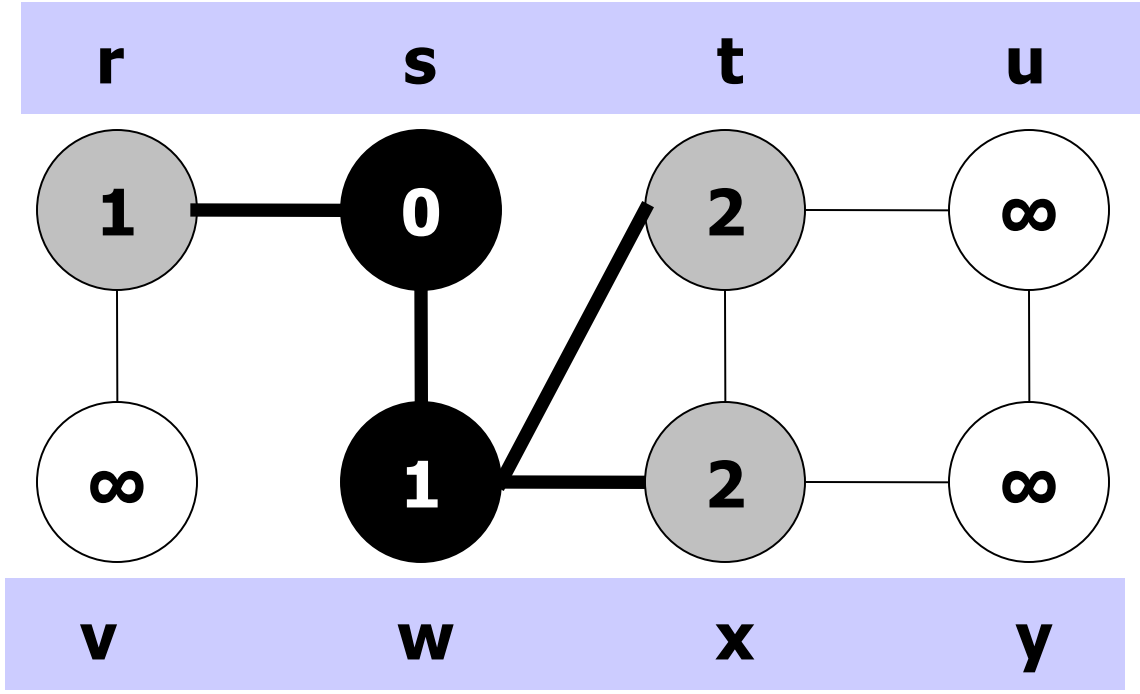
Example 1 (Breadth first search)

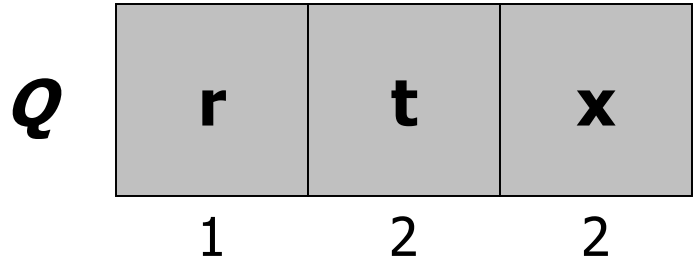


Example 1 (Breadth first search)

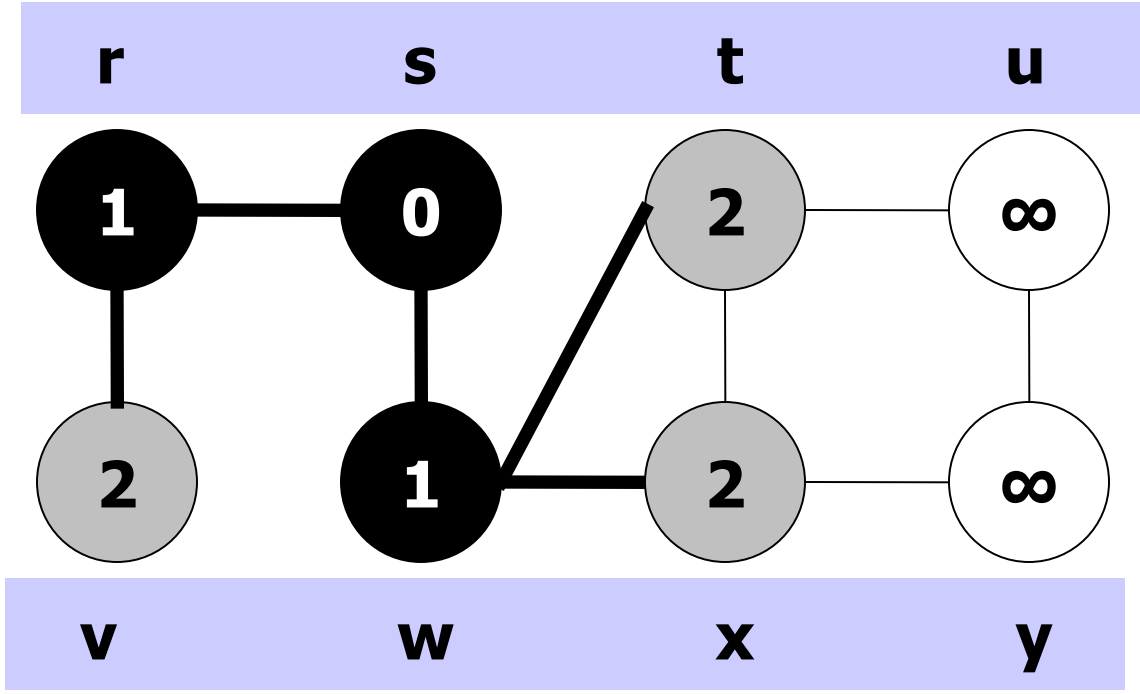


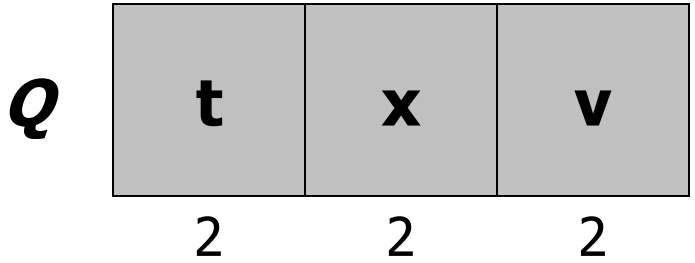
Example 1 (Breadth first search)



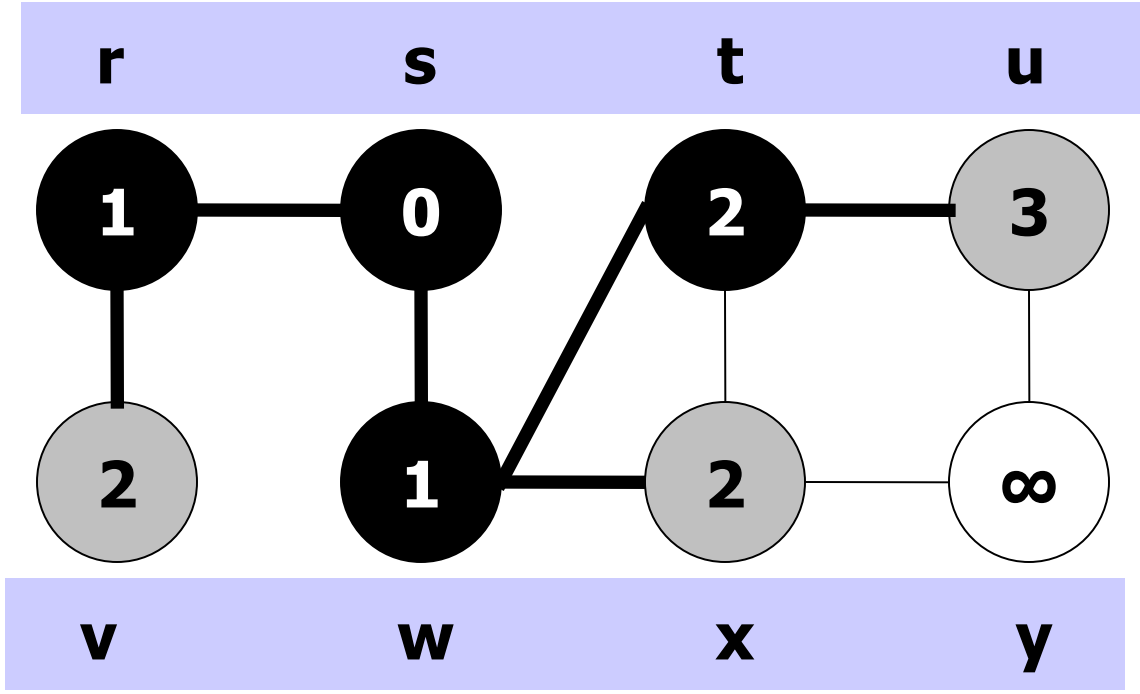


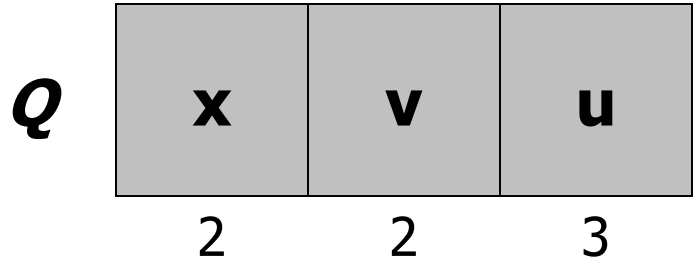
Example 1 (Breadth first search)



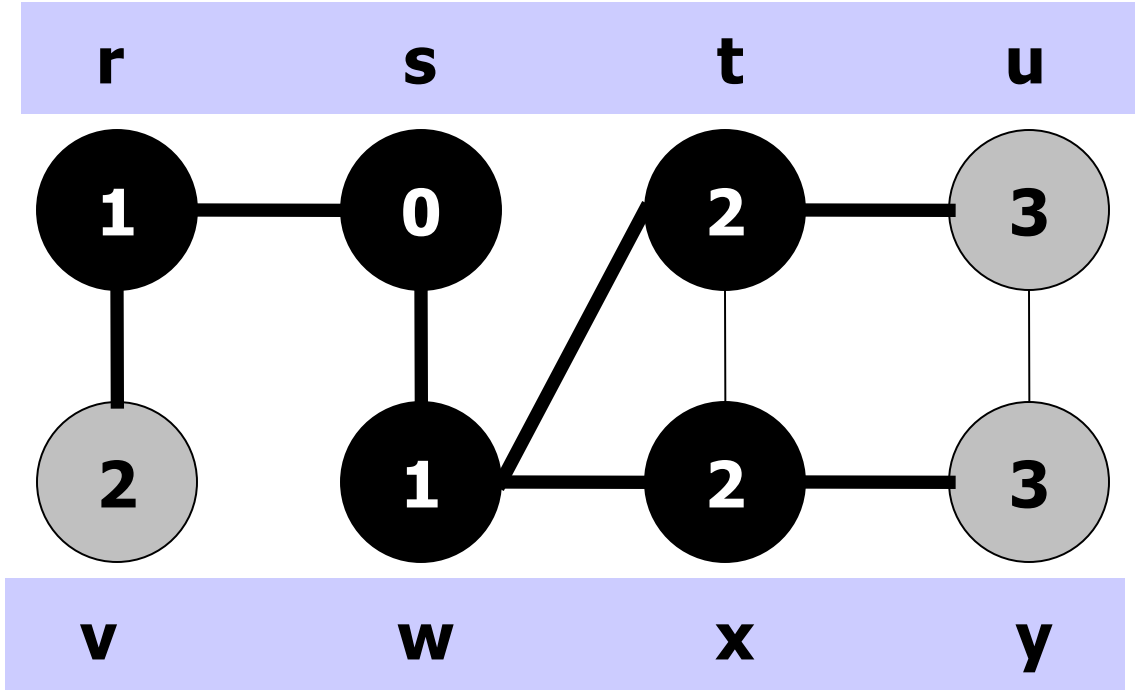


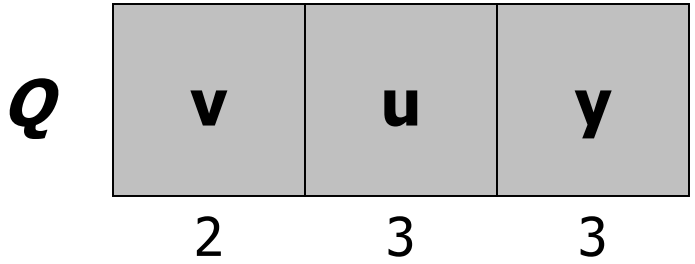
Example 1 (Breadth first search)



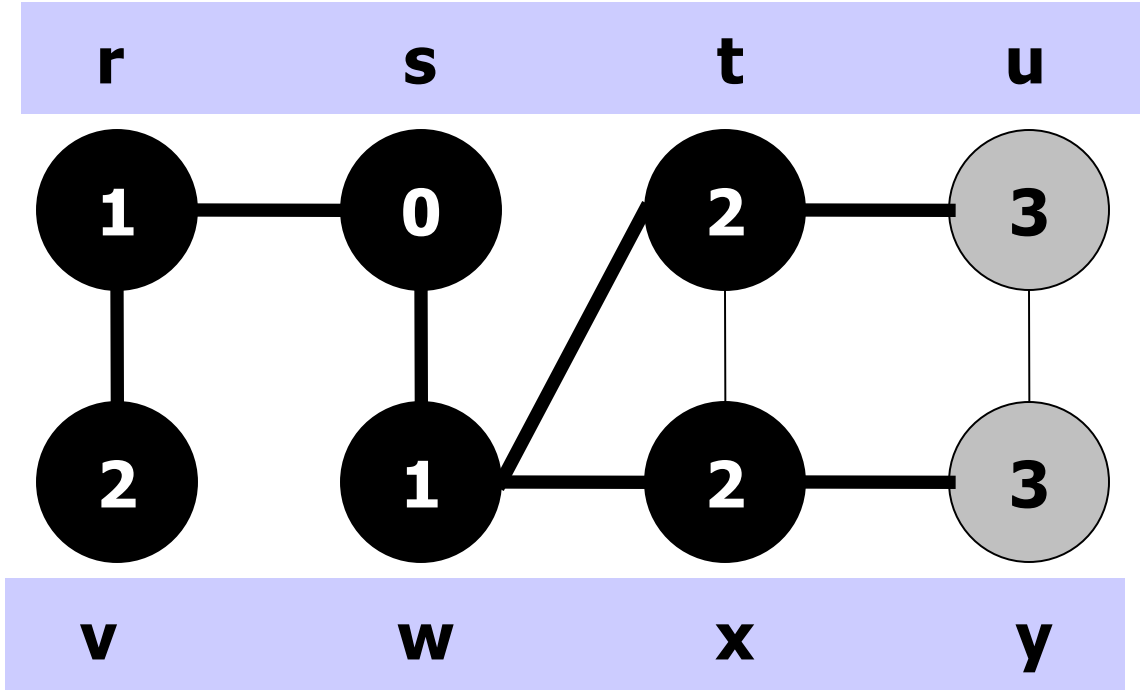


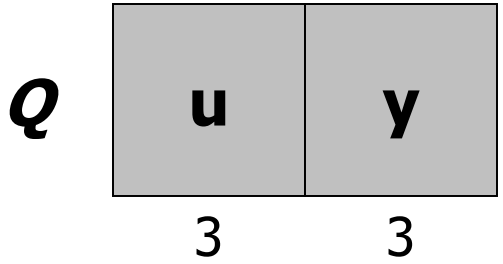
Example 1 (Breadth first search)



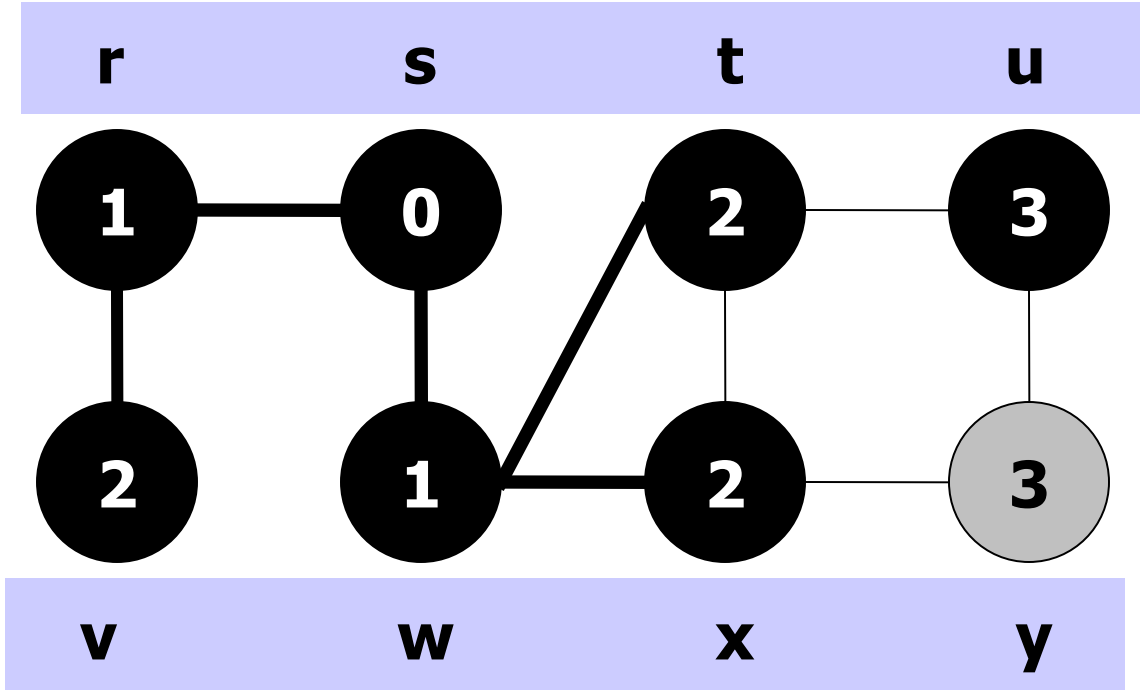


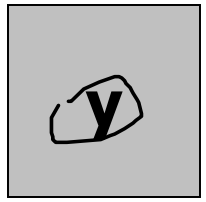
Example 1 (Breadth first search)



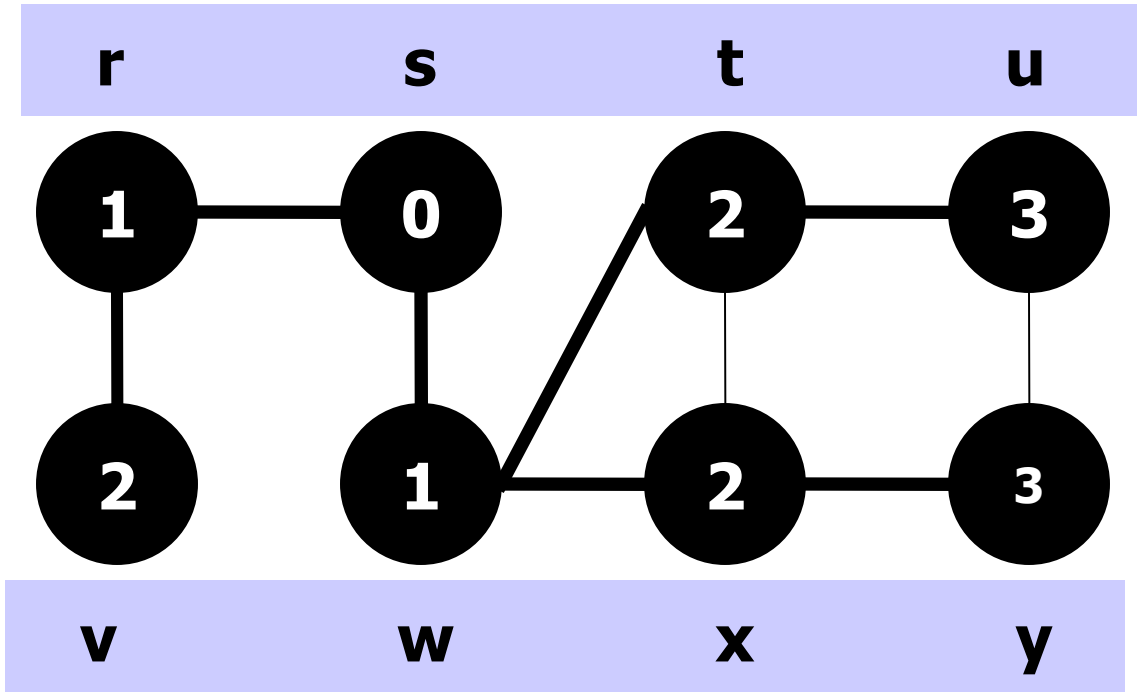


Example 1 (Breadth first search)



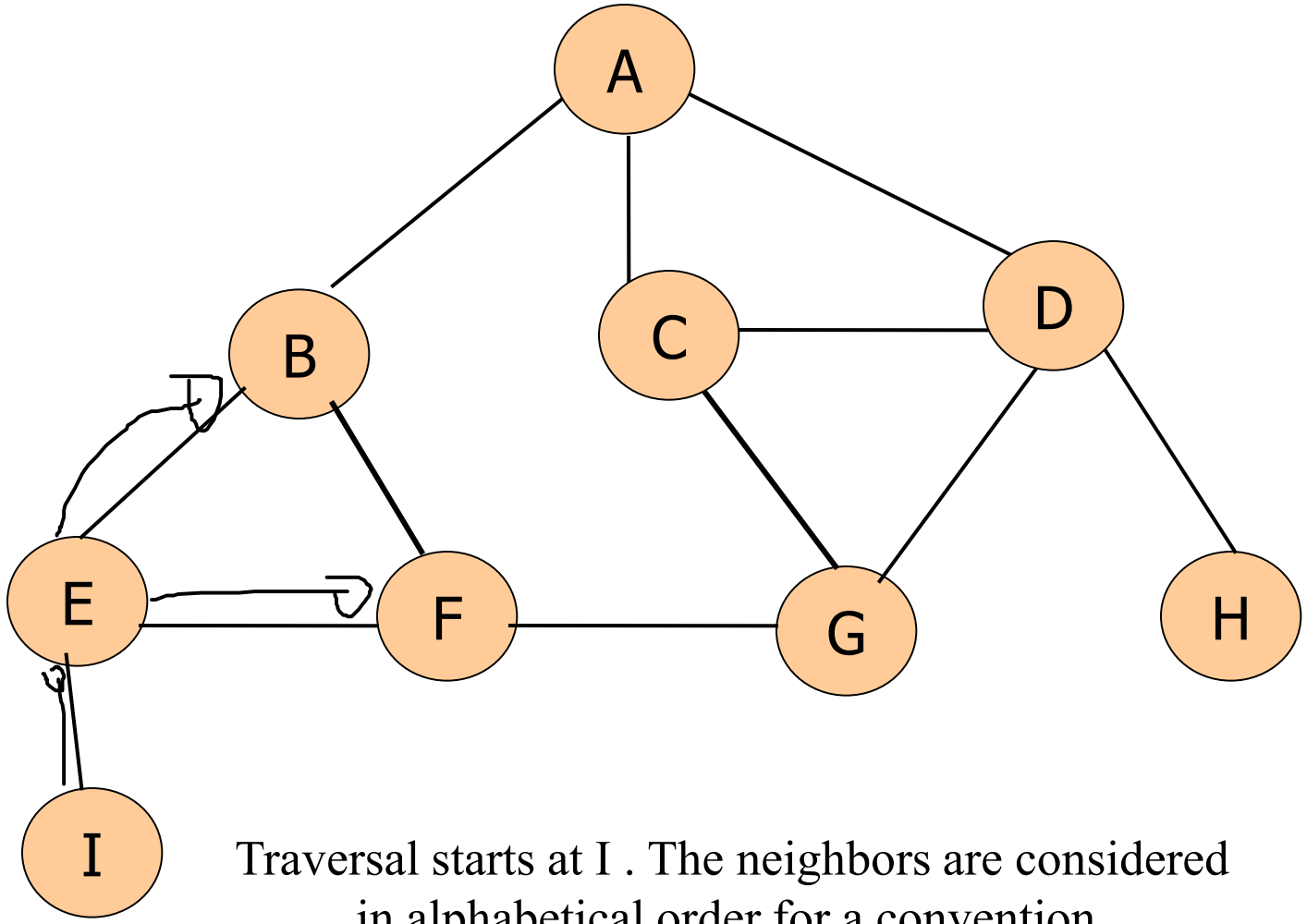
Q 

Example 1 (Breadth first search)



Q Ø

Example 4 (Breadth first search)



Traversal starts at I . The neighbors are considered in alphabetical order for a convention.

Properties of Breadth First Search

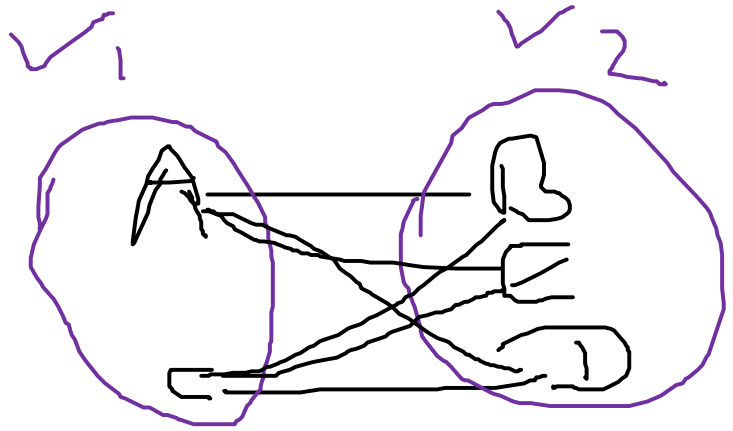
- The running time is O(V+E)
- Space is the bigger problem since it is necessary to keep every node in memory.
- Better at finding shortest path in a graph where the distance is measured by the number of edges.

Bipartite Graph

- A bipartite graph is an undirected graph

  G = (V, E) in which V can be partitioned into two sets V_1 and V_2 such that (u, v) € E implies either
  - u in V_1 and v in V_2 or
  - u in V_2 and v in V_1.

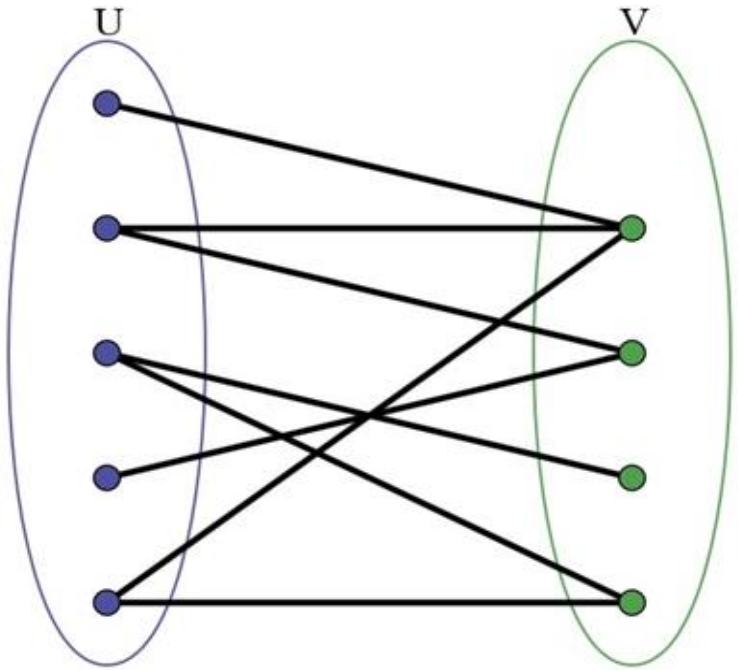
That is, all edges go between the two sets V_1 and V_2.



Bipartite Graph

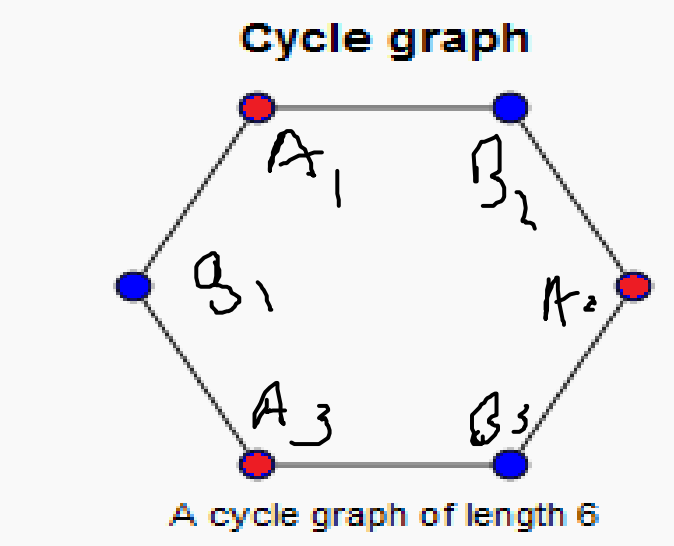
Players

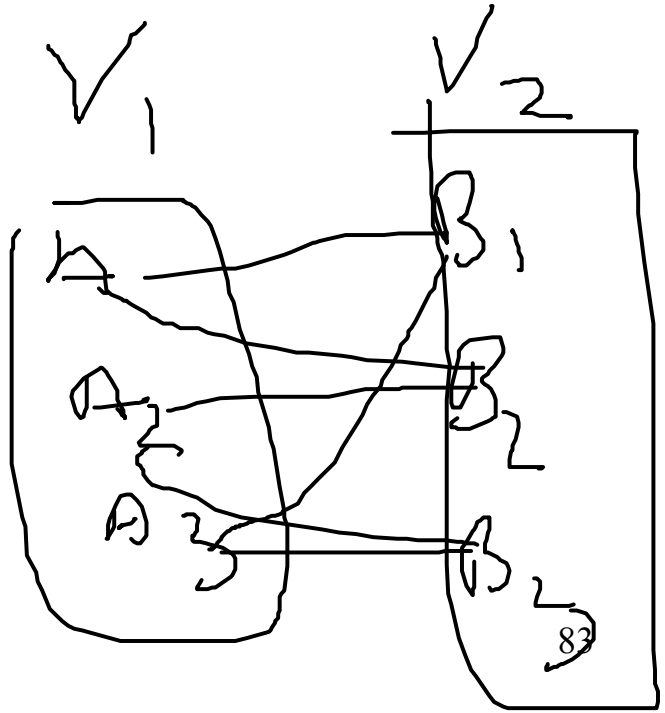
Items



Bipartite Graph

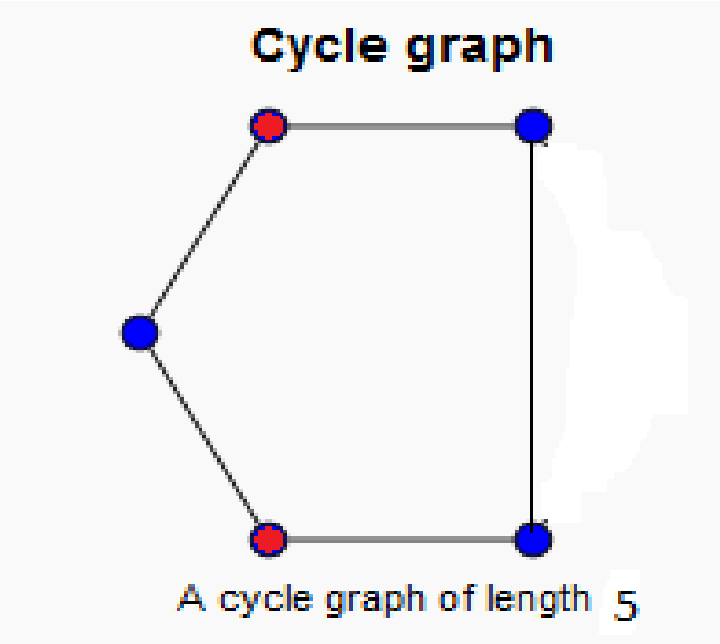
A bipartite graph is possible if the graph coloring is possible using two colors such that vertices in a set are colored with the same color. Note that it is possible to color a cycle graph with even cycle using two colors. For example,



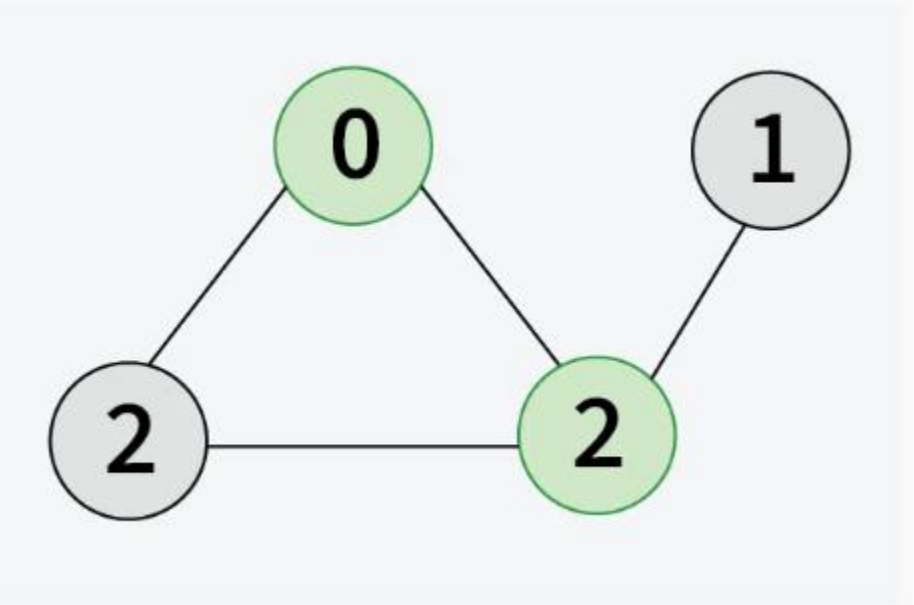


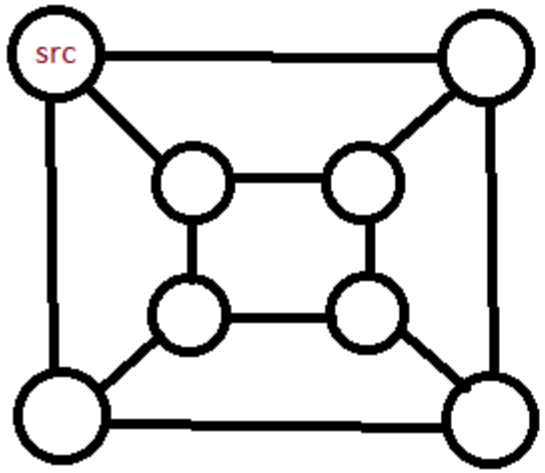
Bipartite Graph

It is not possible to color a cycle graph with odd cycle using two colors.

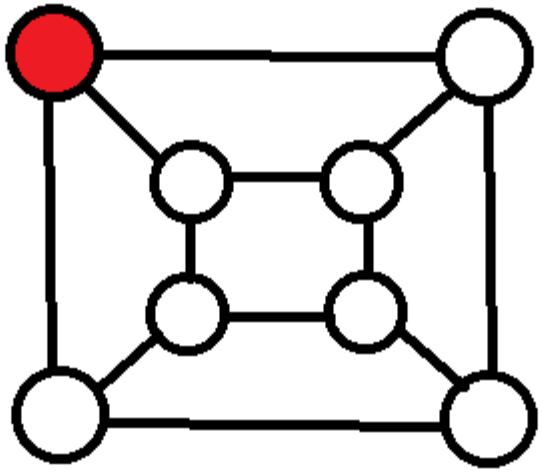


The given graph cannot be colored in two colors such that color of adjacent vertices differs.

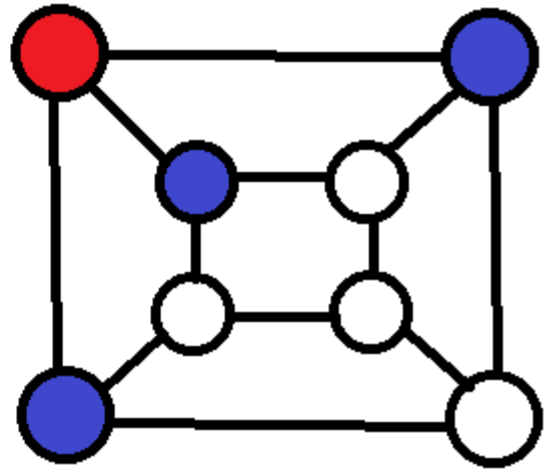




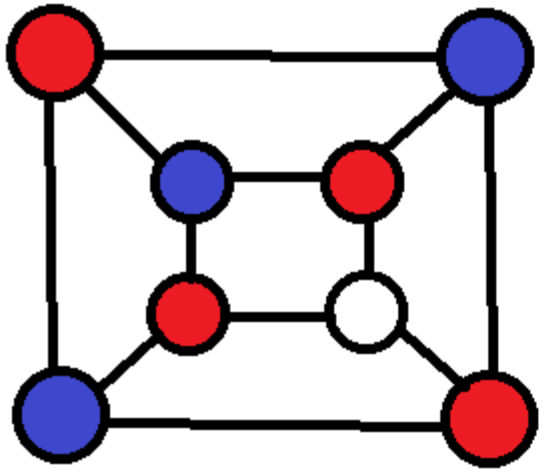
given a graph with source vertex



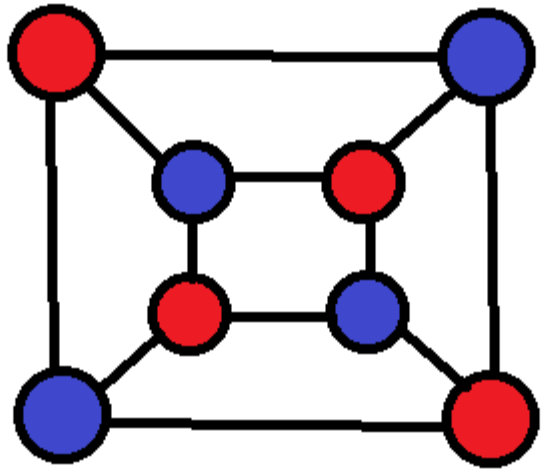
colour src vertex, say red



assign another colour to the neighbours, say blue



assign the neighbours of the vertices of the previous step the colour red



repeat till all vertices are coloured, or a conflicting colour assignment occurs.

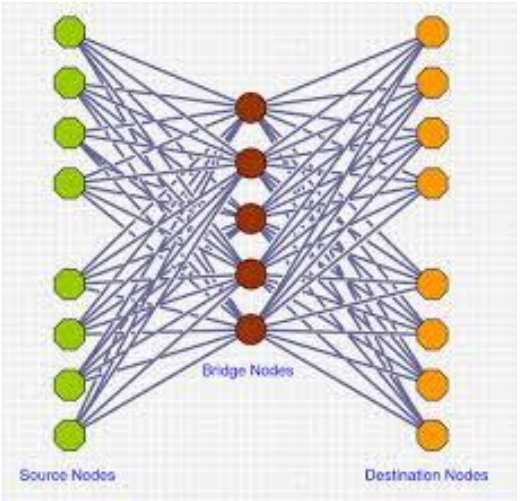
set U: red colour
set V: blue colour

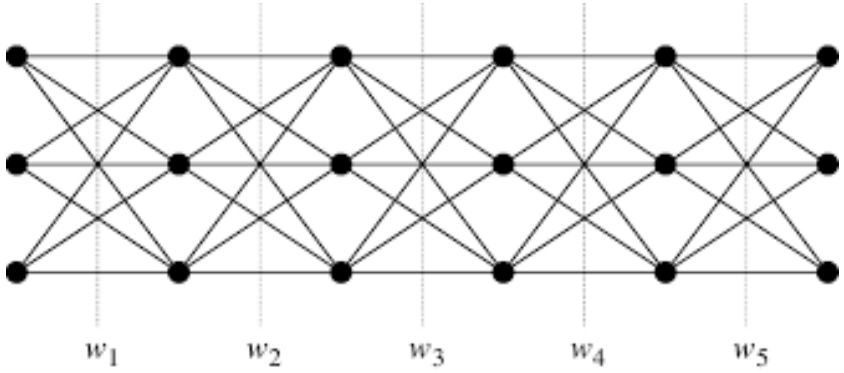
Bipartite Graph

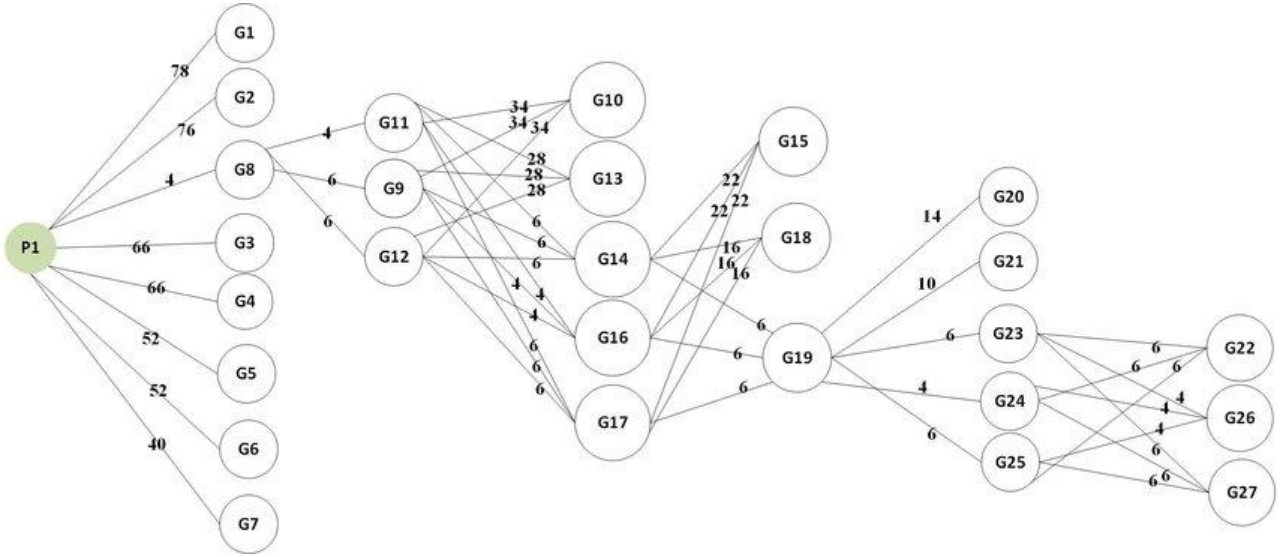
When modeling relations between two different classes of objects, bipartite graphs very often arise naturally. For example,

- Association between football players and clubs
- Schedule of trains and their stops

Multipartite Graphs / k-Partite Graphs

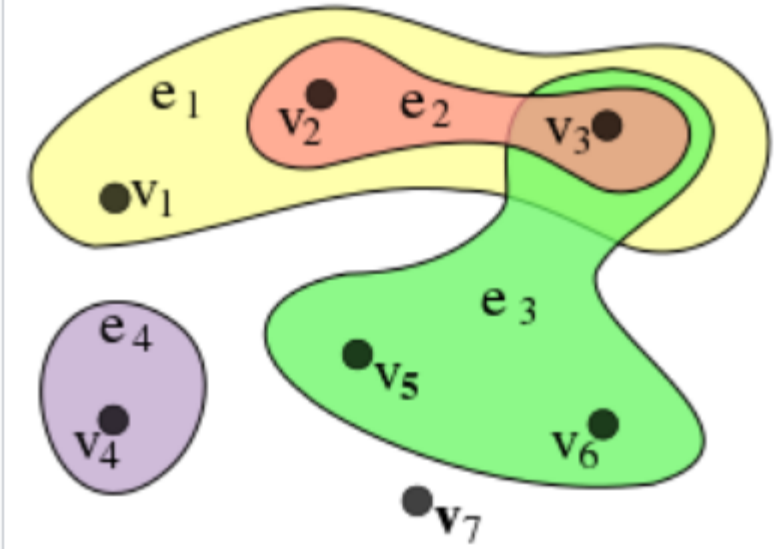






Sources:
https://www.google.com/url?sa=i&url=https%3A%2F%2Fcs.stackexchange.com%2Fquestions%2F104730%2Fall-pair-shortest-path-in-a-tripartite-graph&psig=AOvVaw3MBLsumdFxucjcgSq-miyu&ust=1607651553905000&source=images&cd=vfe&ved=2ahUKEwic0c7vpsLtAhXjxHMBHe67A-4QjRx6BAgAEAc

Hypergraph



An example of a hypergraph, with $X = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$ and $E = \{e_1, e_2, e_3, e_4\} = \{\{v_1, v_2, v_3\}, \{v_2, v_3\}, \{v_3, v_5, v_6\}, \{v_4\}\}$. This hypergraph has order 7 and size 4. Here, edges do not just connect two vertices but several, and are represented by colors.

Source: https://en.wikipedia.org/wiki/Hypergraph

Depth First Search

- Start from an arbitrary node
- Visit *(Explore)* an unvisited adjacent edge
- If the node visited is a *dead end,* go back to the previous node *(Backtrack)*
- Stop when no unvisited nodes are found and no backtracking can be done
- Implemented using a *Stack*

Explore if possible, Backtrack otherwise…

Depth First Search

- *Depth-first search* is another strategy for exploring a graph
  - Explore "deeper" in the graph whenever possible
  - Edges are explored out of the most recently discovered vertex *v* that still has unexplored edges
  - When all of *v*'s edges have been explored, backtrack to the vertex from which *v* was discovered

Depth First Search

- Assume using adjacency lists
- Data structures used:
  - color[u] = stores color of the vertex

    π[u] = predecessor of u

      If u has no predecessor π[u] = null
  - d[u] = discover time array
  - f[u] = finished time array
  - time = timestamp

Depth First Search Algorithm

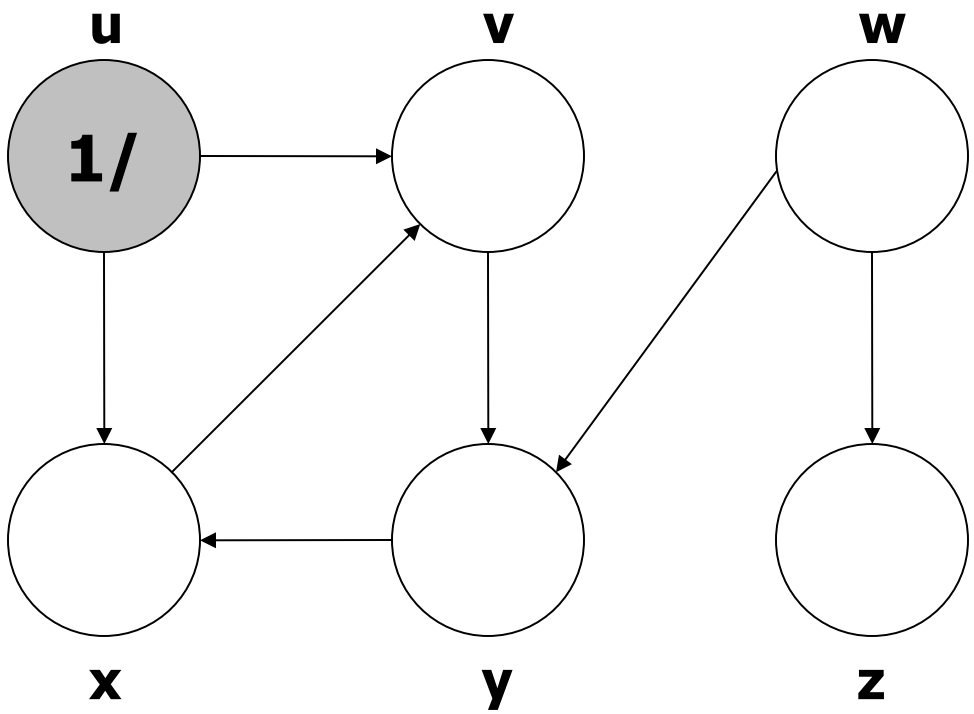
```
DFS(G)
  for each vertex u ∈ V[G]
        do color[u] ← white
         π[u] ← null
  time ← 0
  for each vertex u ∈ V[G]
        do if color[u] = white
                then DFS-Visit(u)
```

Depth First Search Algorithm

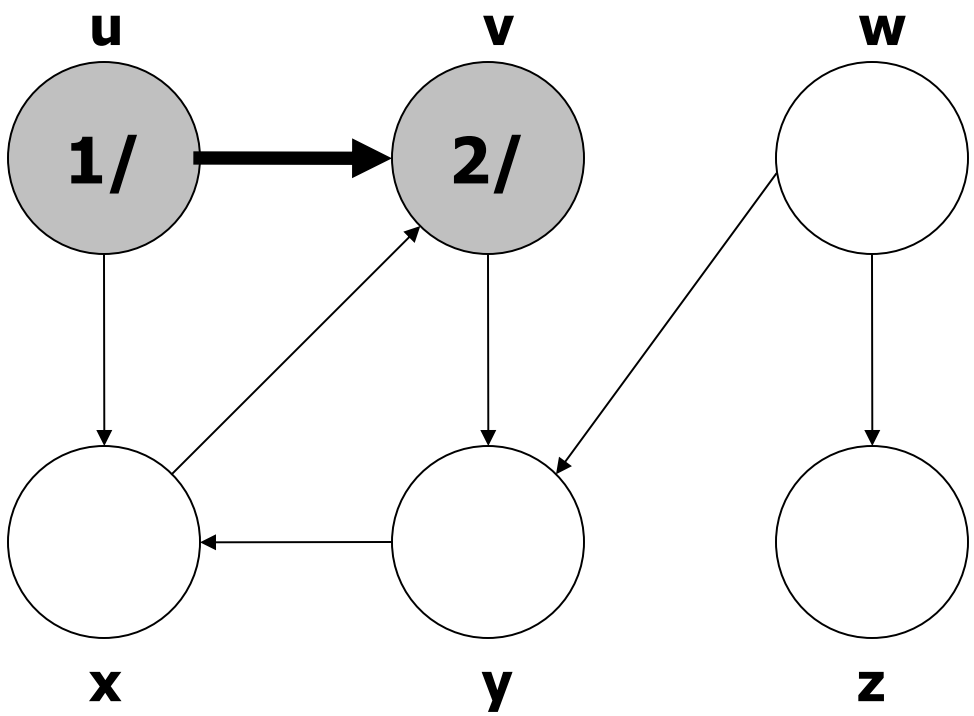
```
DFS-Visit(u)
    color[u] ← GRAY //vertex u has been discovered
    d[u] ← time ← time + 1
    for each v ϵ Adj[u] //explore edge (u,v)
          do if color[v] = WHITE
                    then π[v] ← u
                              DFS-Visit(v)
    color[u] → BLACK //blacken u; it is finished
    f[u] ← time ← time + 1
```

```
Non-Recursive-DFS(G)
    for each vertex u ∈ G(V)
       color[u] ← WHITE
       π[u] ← NIL
     time ← 0
     for each vertex u ∈ G(V)
        if color[u] = WHITE
           color[u] ← GRAY
           time ← time + 1
           d[u] ← time
           push(u, S)
           while stack S not empty
               u ← pop(S)
               for each vertex v ∈ Adj[u]
                   if color[v] = WHITE
                    color[v] = GRAY
                   time ← time + 1
                    d[v] ← time
                    π[v] ← u
                    push(v, S)
      color[u] ← BLACK
      time ← time + 1
      f[u] ← time
```

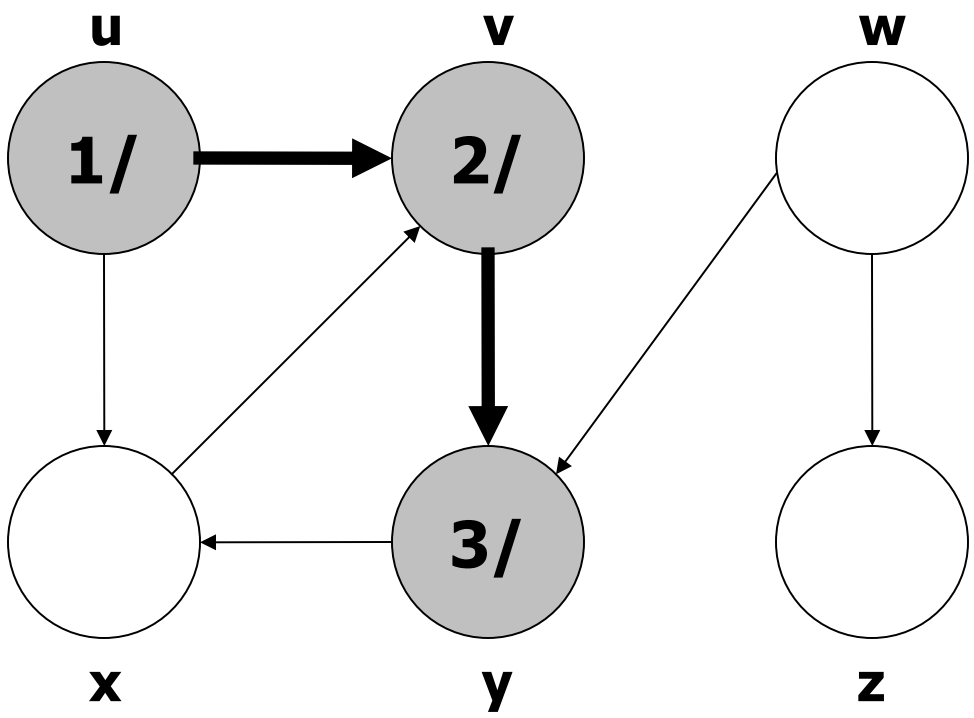
Example 1 (Depth First Search)



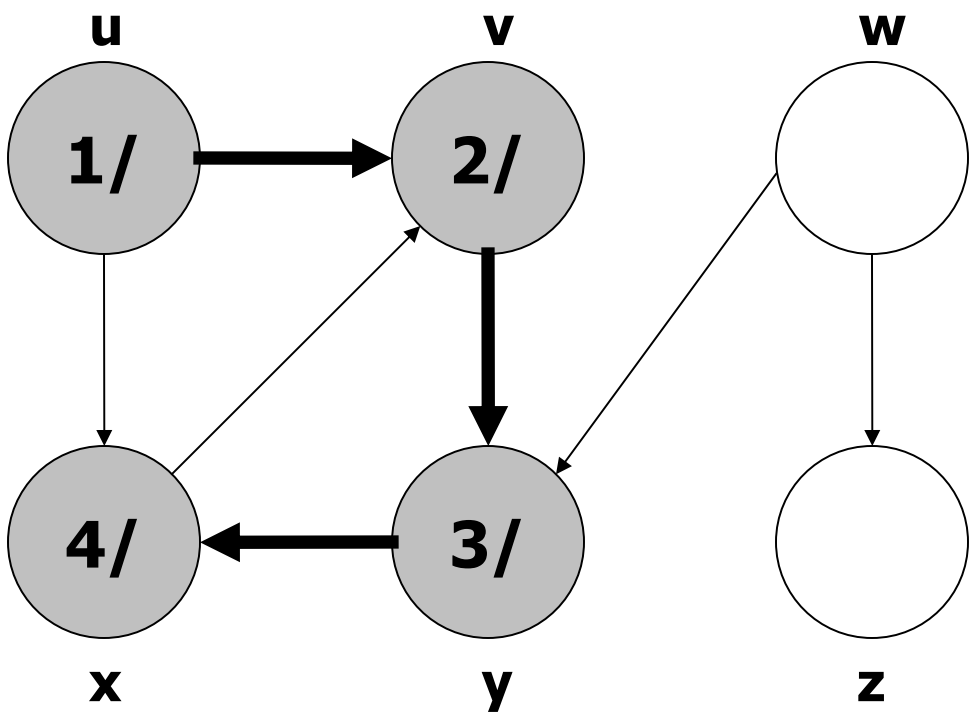
Example 1 (Depth First Search)



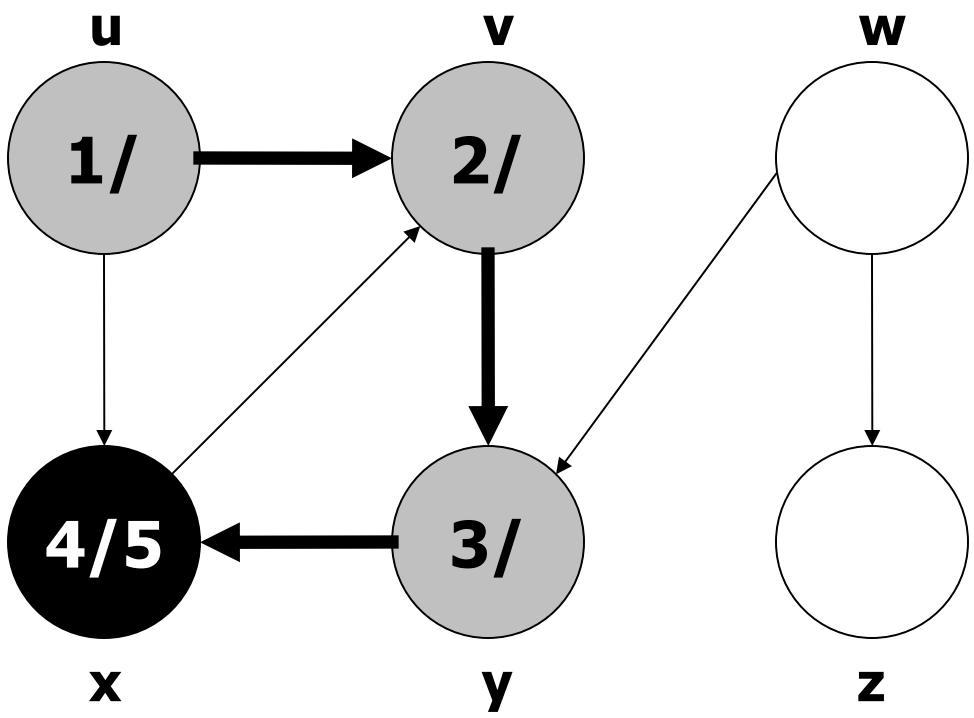
Example 1 (Depth First Search)



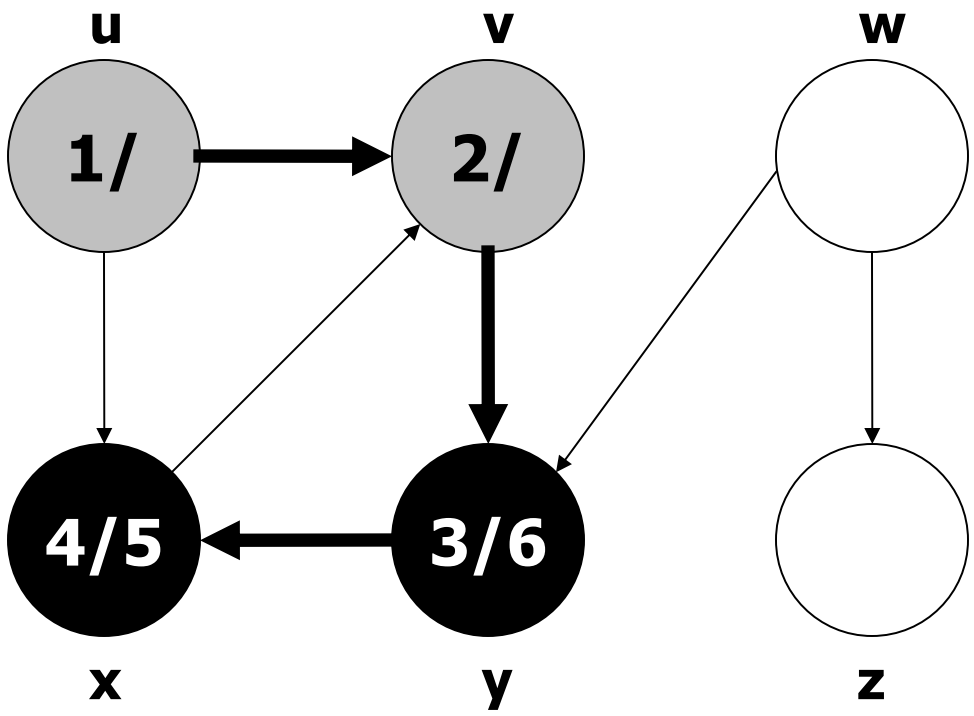
Example 1 (Depth First Search)



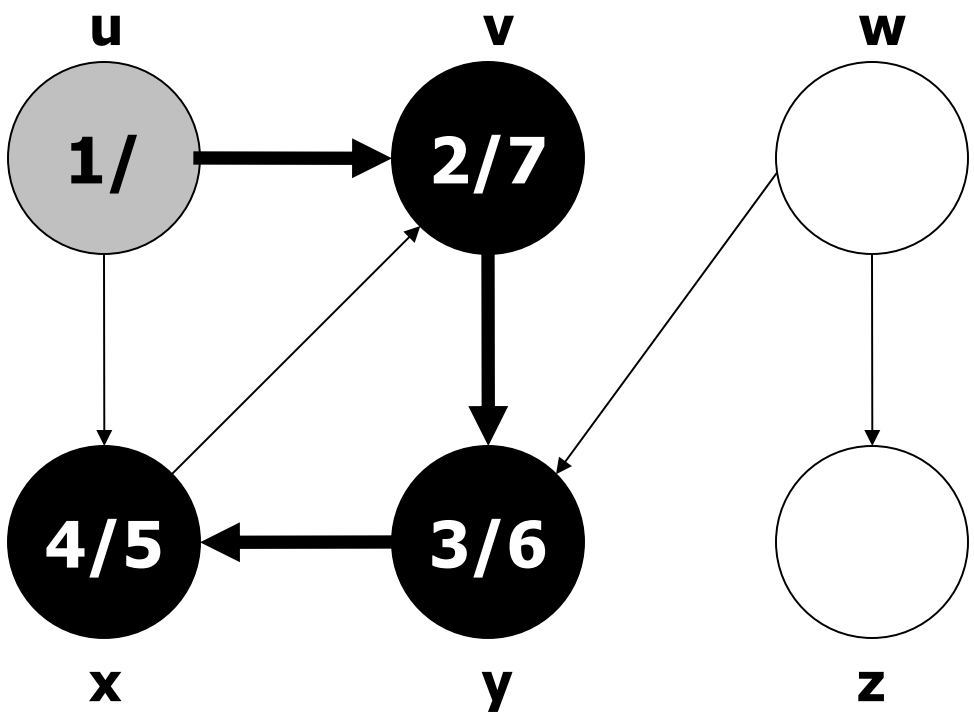
Example 1 (Depth First Search)



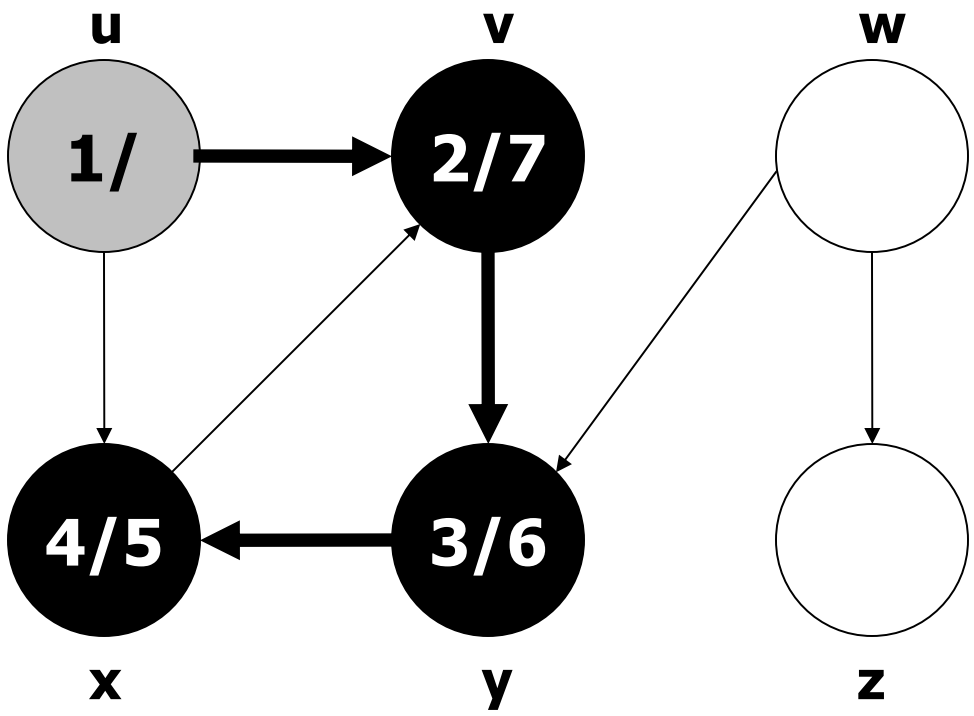
Example 1 (Depth First Search)



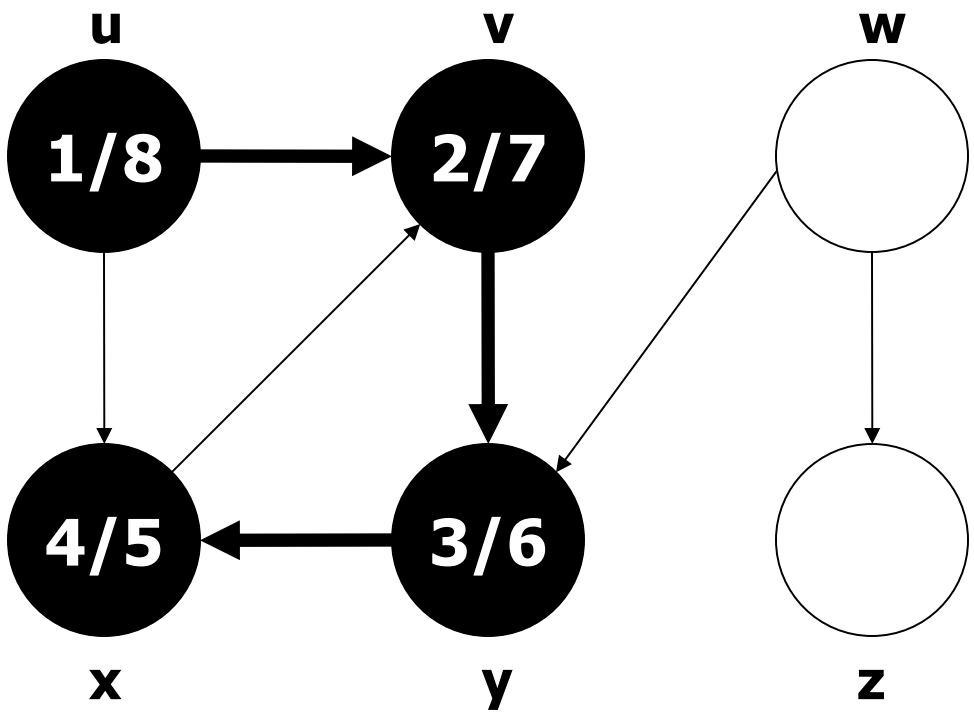
Example 1 (Depth First Search)



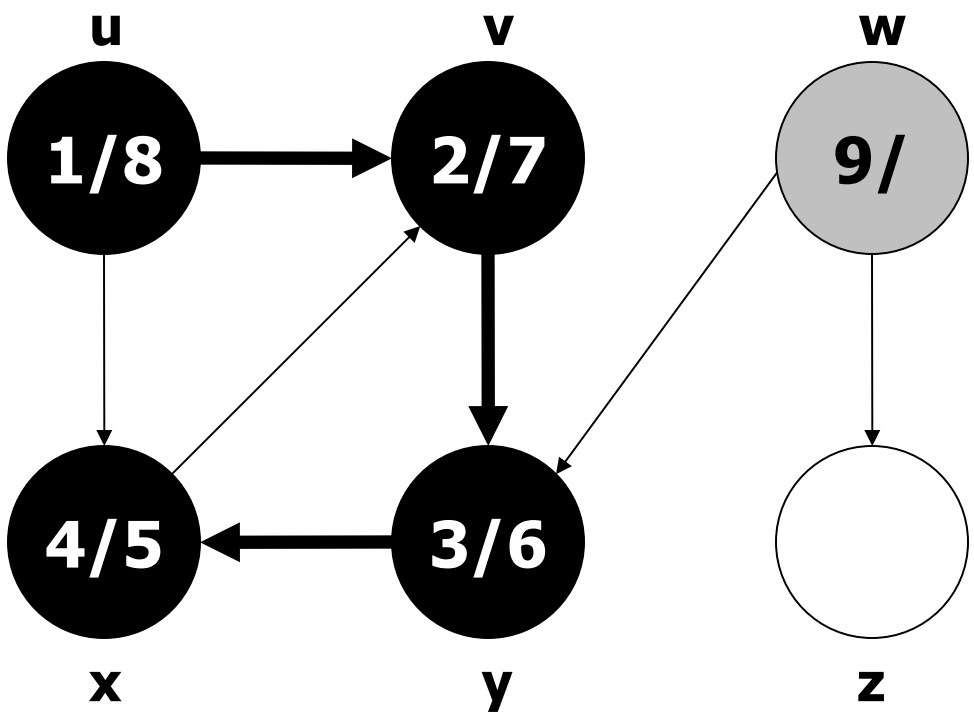
Example 1 (Depth First Search)



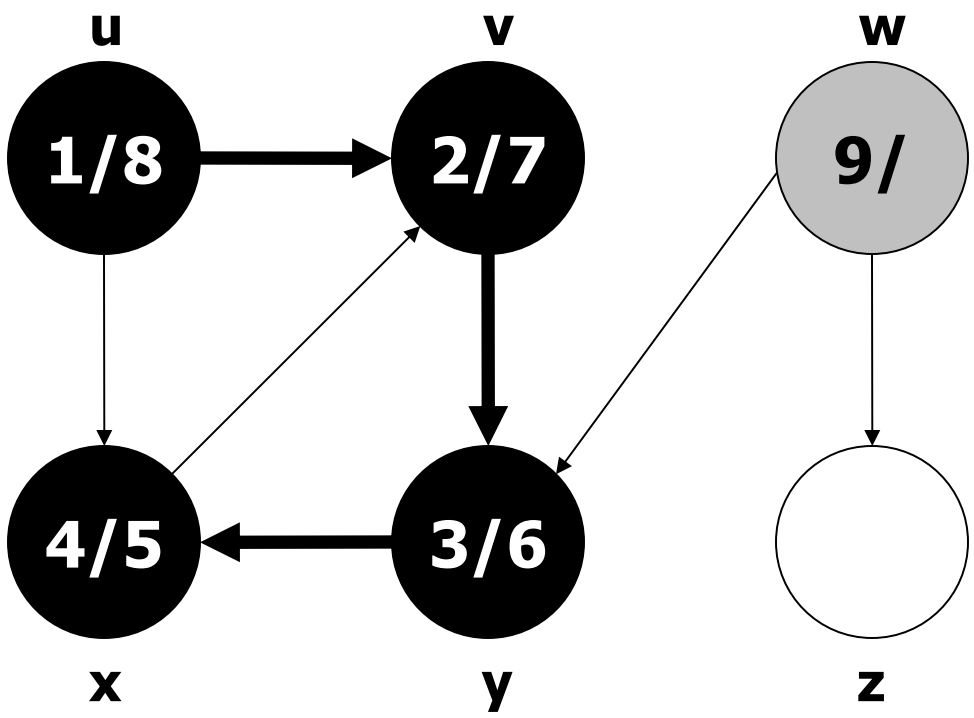
Example 1 (Depth First Search)



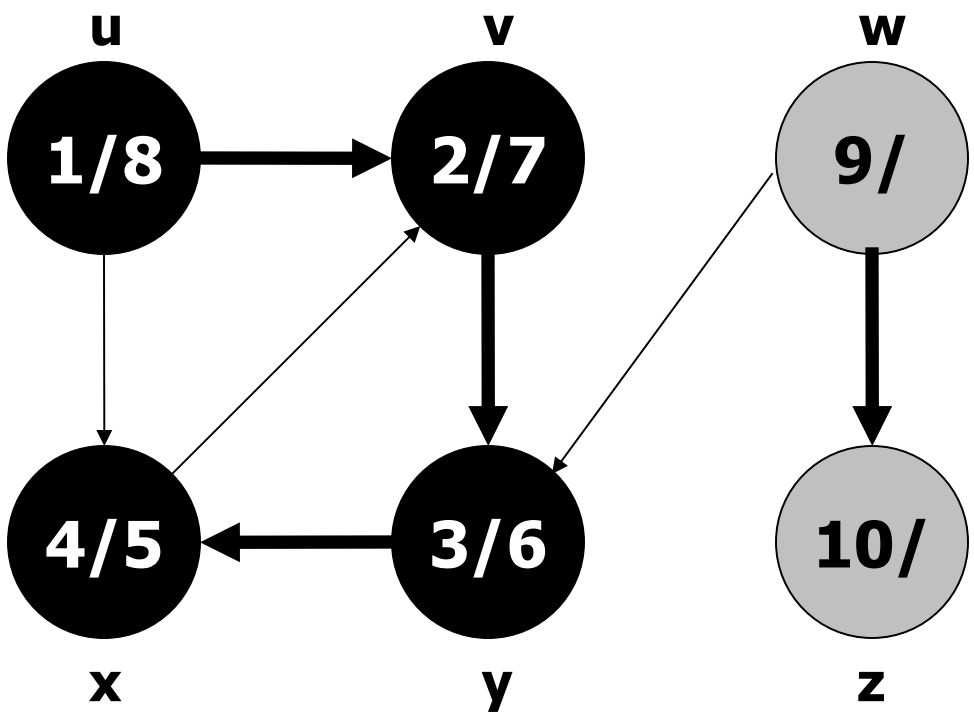
Example 1 (Depth First Search)



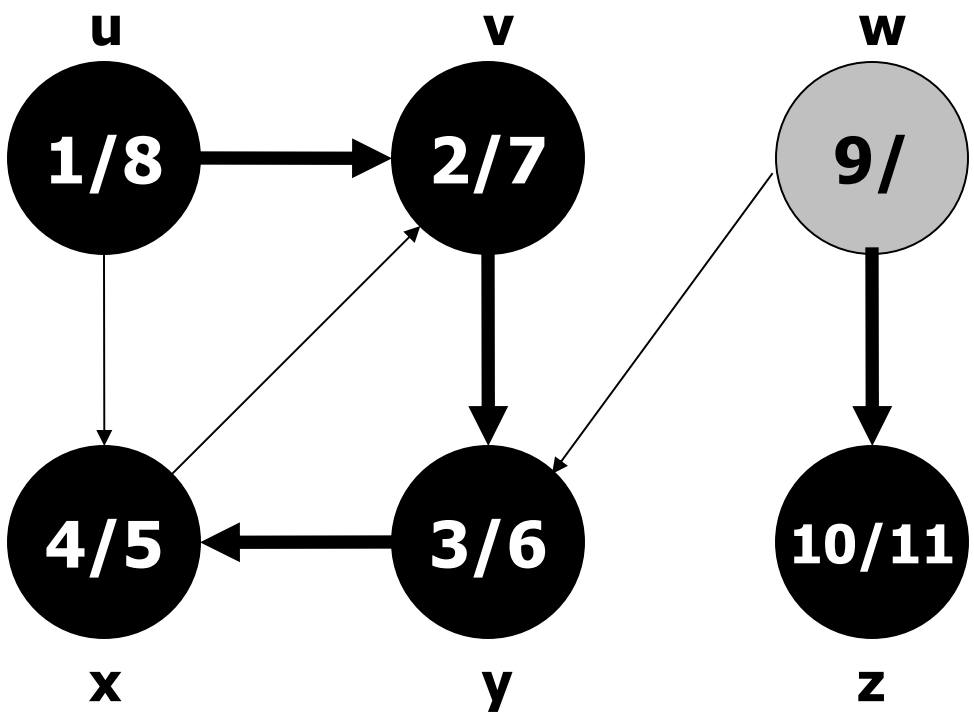
Example 1 (Depth First Search)



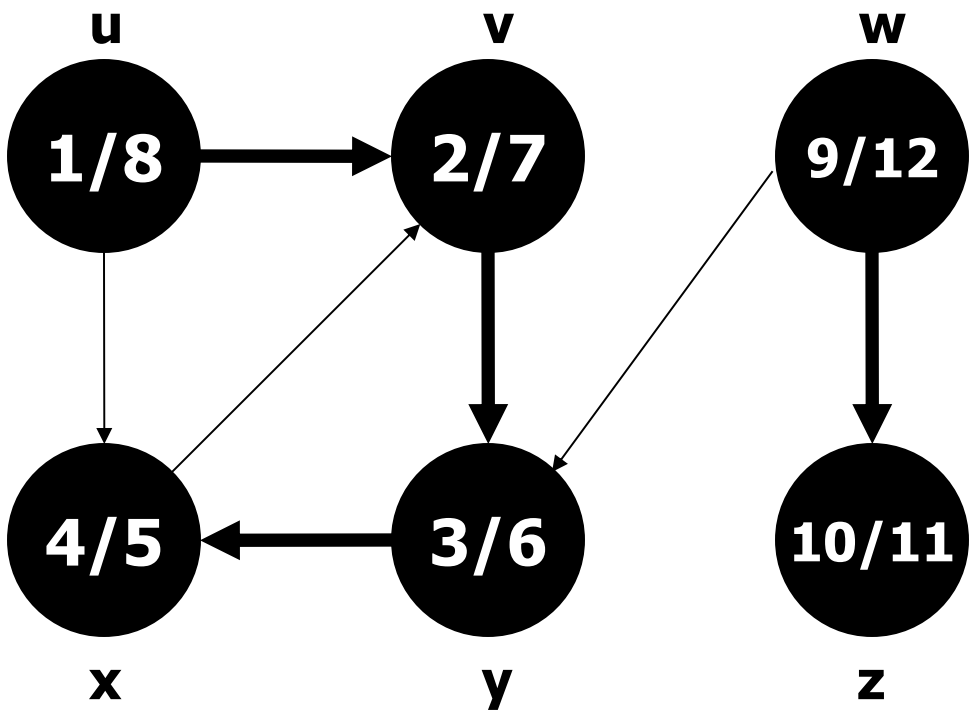
Example 1 (Depth First Search)



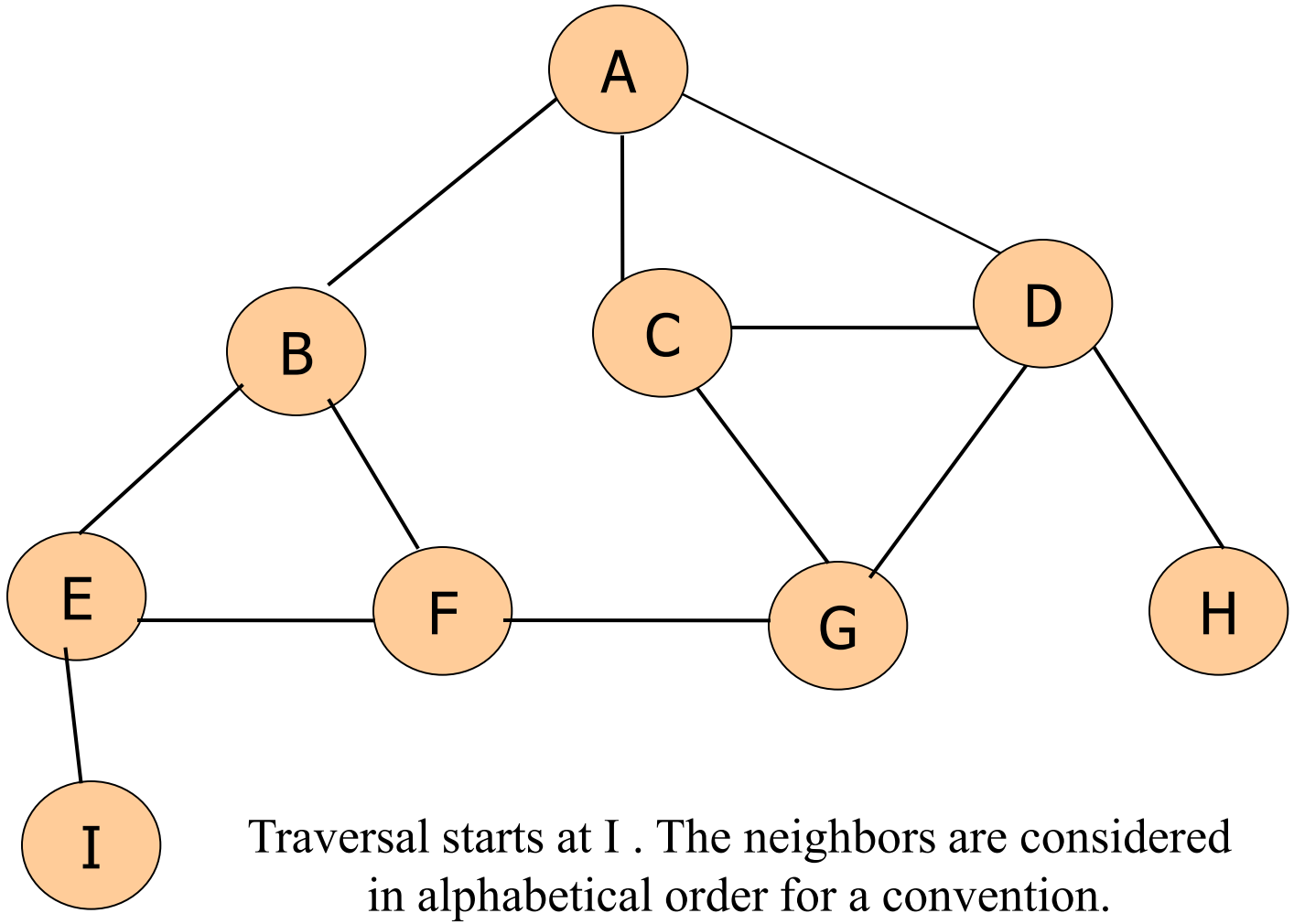
Example 1 (Depth First Search)



Example 1 (Depth First Search)

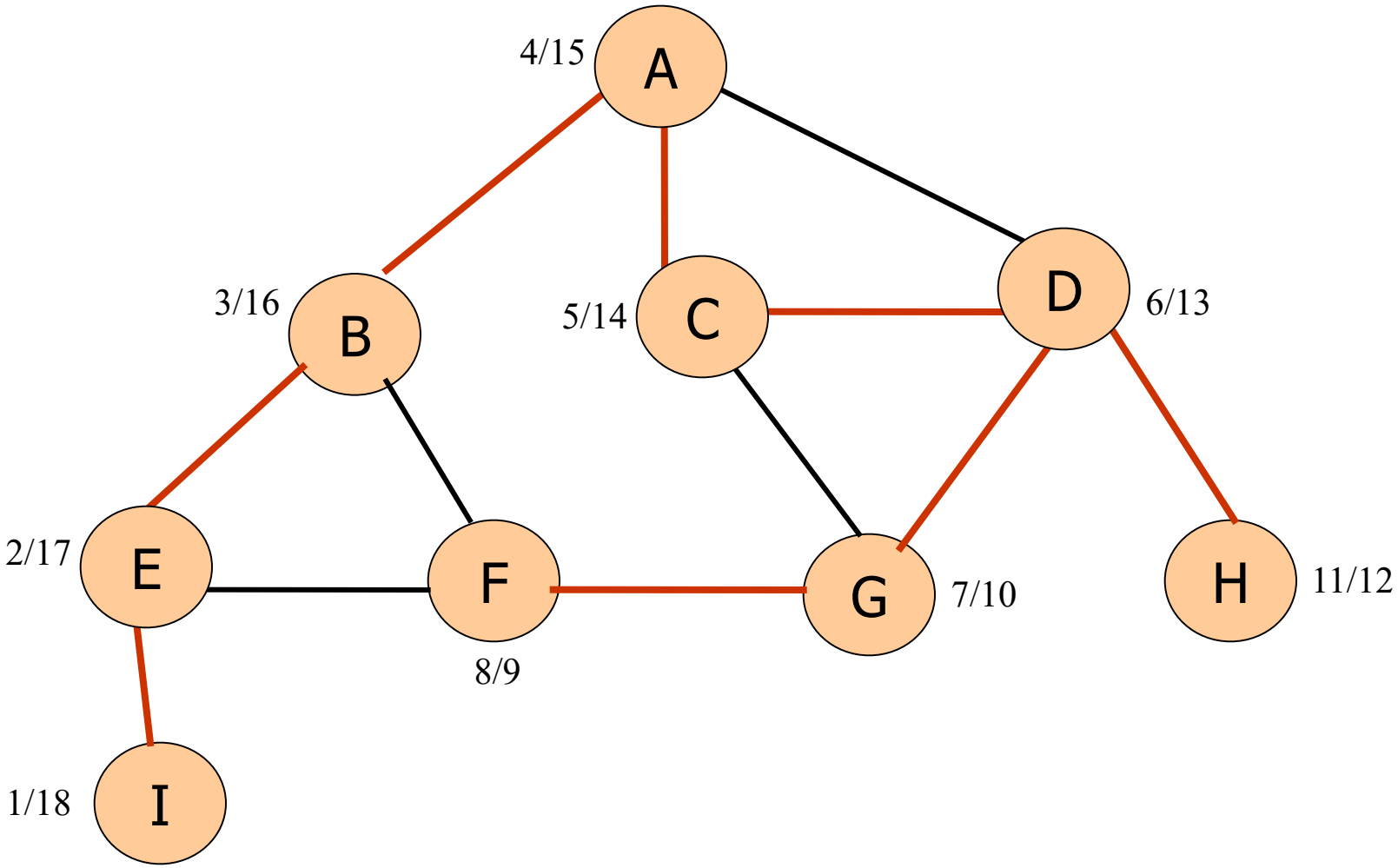


Example 4 (Depth first search)



Traversal starts at I . The neighbors are considered in alphabetical order for a convention.

Example 4 (Depth first search)

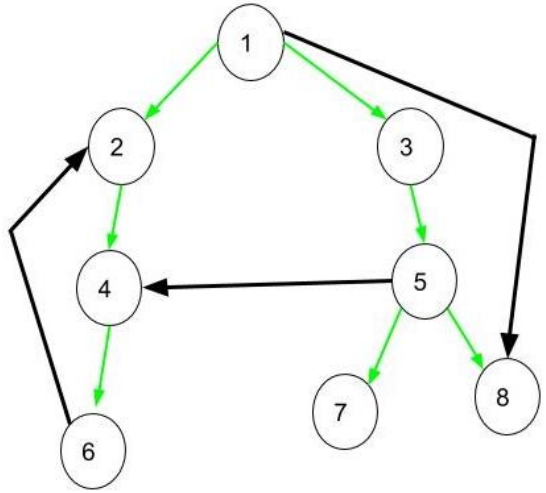


Properties of Depth First Search

- Running time is $O(V+E)$
- Useful for determining whether or not a graph is connected.
- Classification of edges
- Detecting cycles

Classification of Edges

- **Tree Edge**: It is an edge which is present in the tree obtained after applying DFS on the graph. All the Green edges are tree edges.
- **Forward Edge**: It is an edge (u, v) such that v is descendant but not part of the DFS tree. Edge from 1 to 8 is a forward edge.
- **Back edge**: It is an edge (u, v) such that v is ancestor of edge u but not part of DFS tree. Edge from 6 to 2 is a back edge. Presence of back edge indicates a cycle in directed graph.
- **Cross Edge**: It is a edge which connects two node such that they do not have any ancestor and a descendant relationship between them. Edge from node 5 to 4 is cross edge.

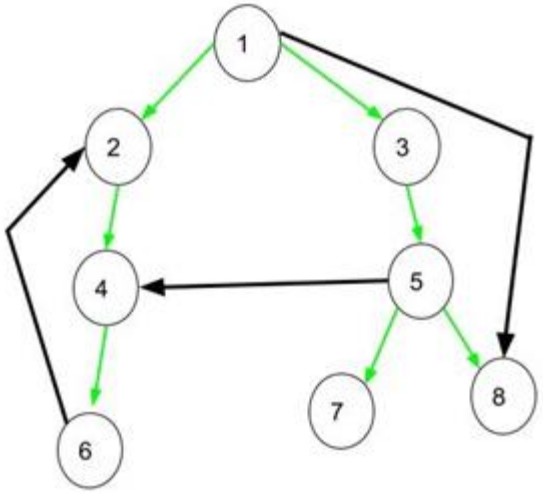


Source: https://www.geeksforgeeks.org/tree-back-edge-and-cross-edges-in-dfs-of-graph/

Edge Classification

DFS-Visit(u)

color[u] ← GRAY //vertex u has been discovered
d[u] ← time ← time + 1
for each v ϵ Adj[u] //explore edge (u,v)
do if color[v] = BLACK
then if d[u] < d[v]
then classify (u,v) as a forward edge
else classify (u,v) as a cross edge
if color[v] = GRAY
then classify (u,v) as a back edge
do if color[v] = WHITE
then π[v] ← u
classify (u,v) as a tree edge
DFS-Visit(v)
color[u] → BLACK //blacken u; it is finished
f[u] ← time ← time + 1



Strongly Connected Components

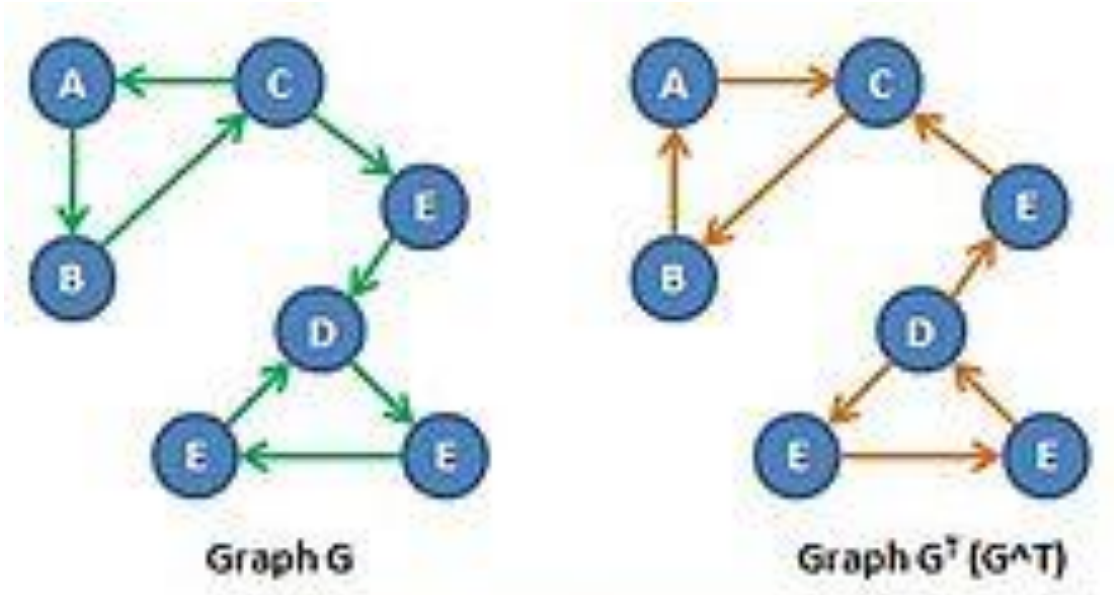
- Definition: the strongly connected components (SCC) C_1, …, C_k of a directed graph G = (V,E) are the largest disjoint sub-graphs (no common vertices or edges) such that for any two vertices u and v in C_i, there is a path from u to v and from v to u.
- Goal: compute the strongly connected components of G in linear time (|V|+|E|).

Strongly Connected Components

- The algorithm for finding strongly connected components of G = (V, E) uses the transpose of G, which is defined as:

  $G^T = (V, E^T)$, where $E^T = \{(u, v): (v, u) \text{ in } E\}$

  G^T is G with all edges reversed.



Graph G and its transpose G^T

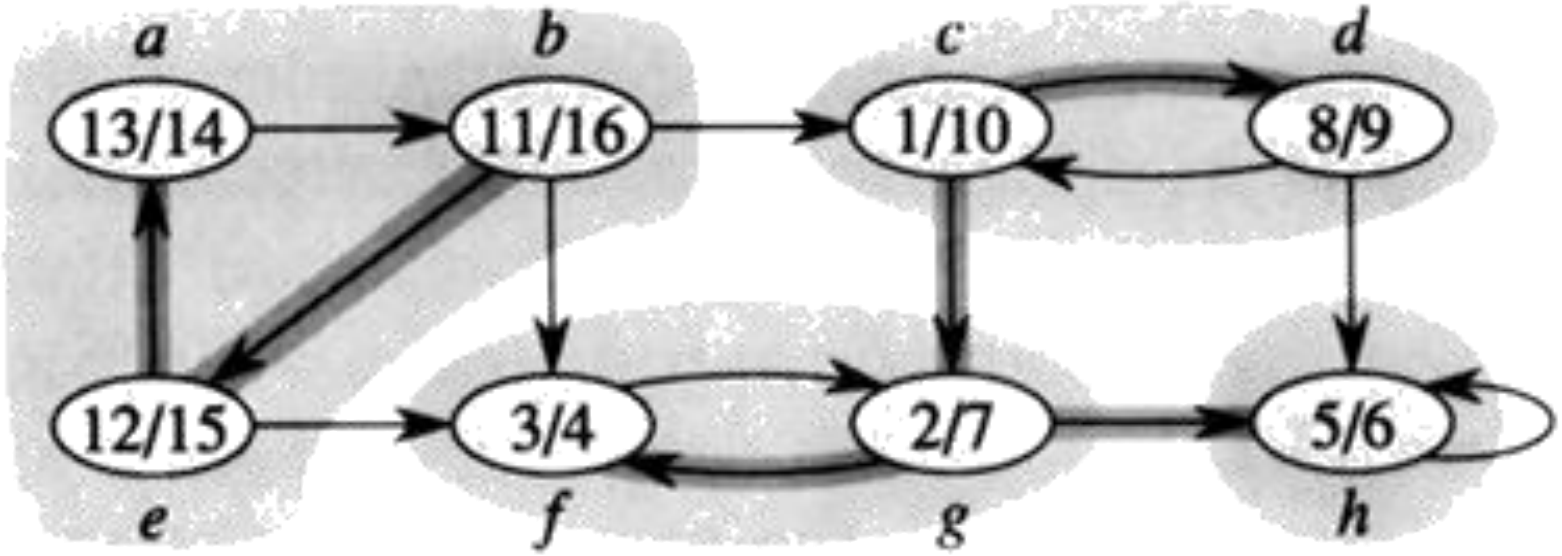
STRONGLY-CONNECTED-COMPONENTS (G)

1. **Call** DFS(G) to compute finishing times f[u] for all *u*.
2. **Compute** G^T
3. **Call** DFS(G^T), considering vertices in order of decreasing f[*u*] (as computed in first DFS)

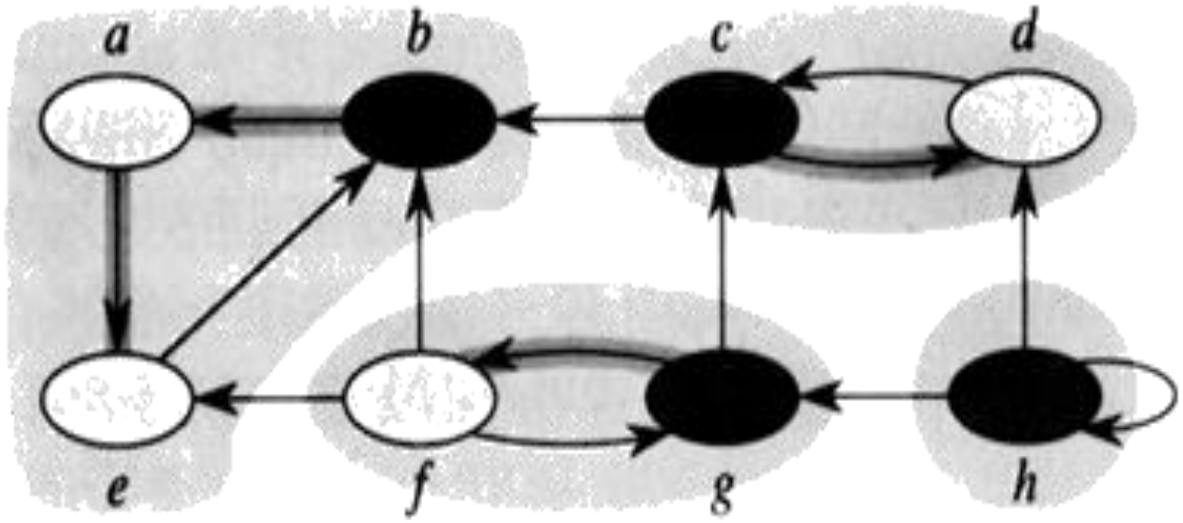
   If the DFS does not reach all vertices, start the next DFS from the highest numbered remaining vertex.

4. **Output** the vertices in each tree of the depth-first forest formed in second DFS.

Call DFS(G)



Compute G^T & Call DFS(G^T)

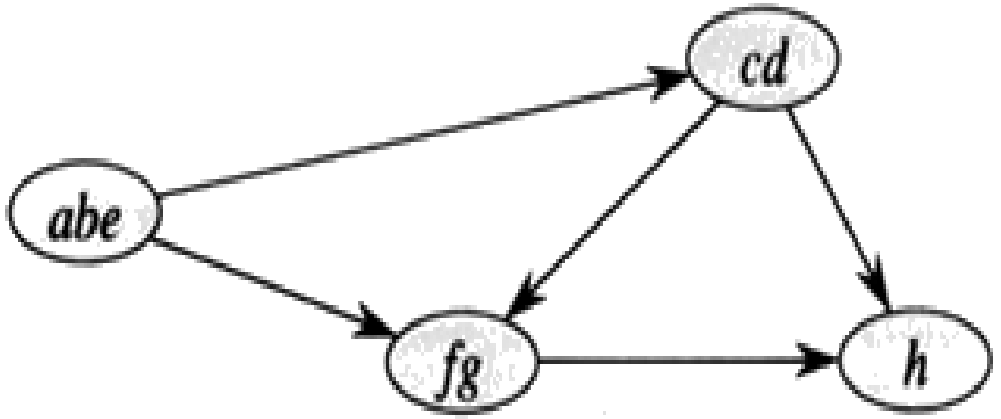


The graphs G and G^T have the same SCC . This means that vertices *u* and *v* are reachable from each other in G if and only if reachable from each other in G^T.

Output the vertices of each tree in the DFS-forest

The graph has 4 strongly connected components: C_1, C_2, C_3 and C_4.

c1= {*a*, *b*, *e*},c2= {*c*, *d*}, c3={*f*, *g*}, and c4={*h*}



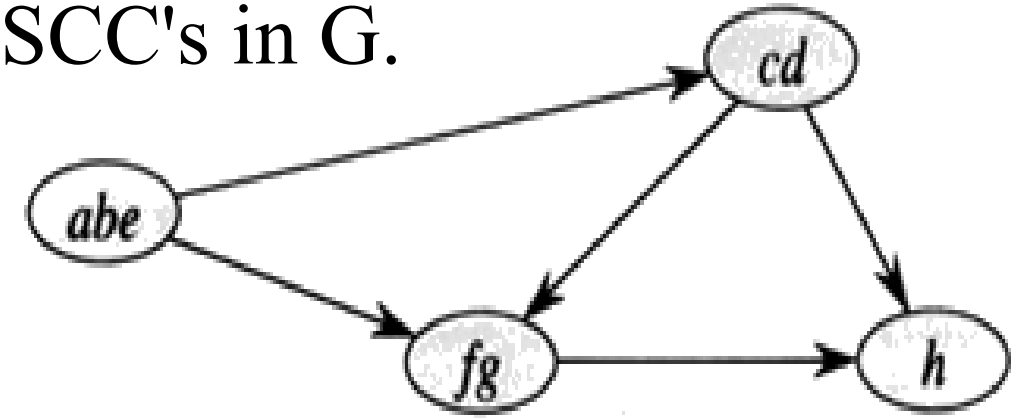
Strongly Connected Components

- If G has an edge from some vertex in C_i to some vertex in C_j where $i \neq j$, then one can reach any vertex in C_j from any vertex in C_i but not return.
- In the example, one can reach any vertex in C_2 from any vertex in C_1 but cannot return to C_1 from C_2.

Component Graph

Key property of the component graph is defined as follows:

- $G^{SCC} = (V^{SCC}, E^{SCC})$, where V^{SCC} has one vertex for each SCC in G and E^{SCC} has an edge if there's an edge between the corresponding SCC's in G.

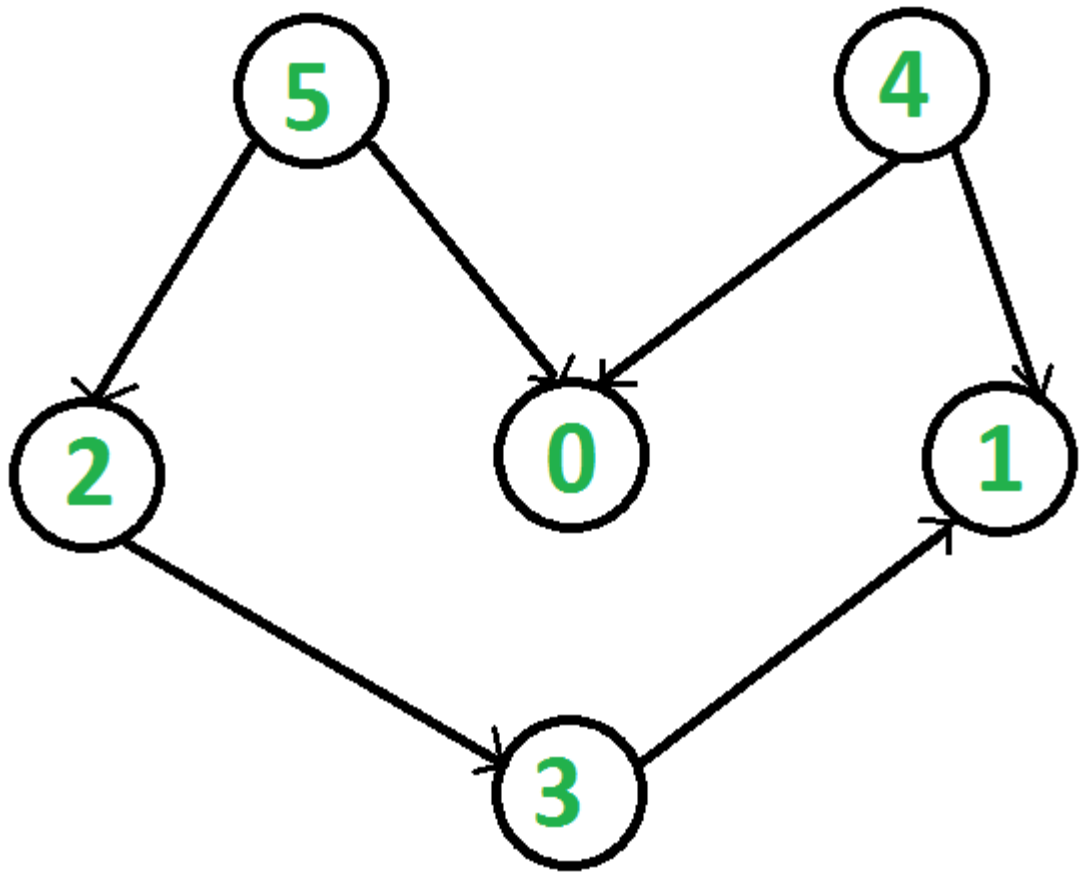


The key property of G^{SCC} is that the component graph is a dag (Directed acyclic graph).

Topological Sort

Topological _Sort(G)

1. Call DFS(G) to compute finishing times f[v] for each vertex v
2. As each vertex finished, insert it into the front of a linked list
3. Return the linked list of vertices


5
4
2
0
1
3